ANQI: An Approximate Reverse k-NN Query Index at Scale

Yujian Fu*
National University of Singapore
yujianfu@comp.nus.edu.sg

Weida Liang*
National University of Singapore
weida_liang@u.nus.edu

Anqi Dang*
Nanyang Technological University
anqi012@e.ntu.edu.sg

Weng-Fai Wong
National University of Singapore
dcswwf@nus.edu.sg

Bingsheng He
National University of Singapore
dcsheb@nus.edu.sg

***Abstract*—Approximate reverse k-nearest-neighbor (RkNN) search retrieves the base vectors that would rank a query vector among their own k nearest neighbors. A base vector accepts a query only if the query beats a cutoff set by its own neighborhood, which creates two costs at scale. First, nearby vectors can reject a query that farther ones accept, so routing through a proximity index must either expand many candidates or miss true reverse neighbors. Second, the cutoff differs across base vectors and ranks, so it must be either recomputed for every query or stored in a large vector-by-rank cutoff table.**

We present ANQI, an RkNN index for Euclidean distance and inner product that reduces both costs. ANQI incorporates each vector's reverse cutoff into graph routing, allowing a standard ANN graph to generate high-recall candidates with less work than proximity routing. A compact verifier then checks these candidates at the requested rank without storing the full vector-by-rank cutoff table. One graph and verifier serve every rank materialized at build time, so the index does not need to be rebuilt for each k. Across five million-scale datasets, ANQI is at least 2.51× faster than the fastest baseline with the same recall (0.95 for Euclidean distance, 0.99 for inner product) at comparable precision. By sharing one graph and a compact verifier across ranks, this design also scales to a billion-vector setting. To our knowledge, ANQI is the first billion-scale RkNN index, and it supports SIFT1B queries at $k = 10$ with recall $.98$, precision $.97$, and $17\,\mathrm{k}$ QPS. The implementation is available at github.com/Yujian-Fu/anqi-implementation.

***Index Terms*—reverse k-nearest neighbor, vector search, proximity graph, approximate nearest neighbor, inner product search**

I. Introduction

Conventional k-nearest-neighbor (kNN) search is query-centered: given a vector q, it returns the k base vectors closest to q. Reverse k-nearest-neighbor (RkNN) search inverts this direction and asks whose own top-k list would contain q. It therefore supports item-driven retrieval, where a new item rather than a user starts the search. For example, a seller can find the users most likely to rank a new or discounted item highly [1], [2], and a service can find subscribers for whom a new document is relevant, or customers affected by a new store location [3]. Fig. 1 contrasts these two query directions.

*The first three authors contributed equally to this work.

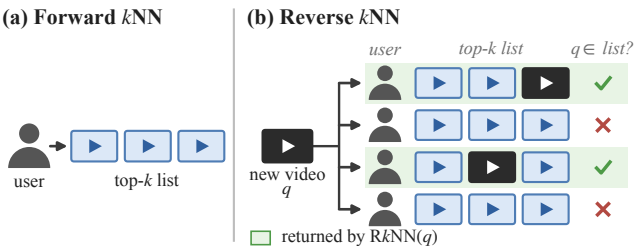


Fig. 1. Forward kNN retrieves videos for one user. Given a new video q, RkNN instead returns the users whose own top-k lists would include it.

On modern embedding collections, however, the reverse query is costly for two reasons. First, forward proximity does not order reverse candidates. Each base vector judges q against its own neighborhood, so a nearby vector can reject q while a farther one accepts it. A proximity index must therefore expand many candidates to reach the true reverse neighbors, and it can still miss some even when it retrieves an accurate forward neighborhood. Section V-F confirms this overhead: to reach the same recall, routing by distance alone needs more graph work than routing that accounts for the cutoff, and the gap widens as the recall target rises. Second, a base vector accepts q only if q beats a cutoff set by its own neighborhood, and this cutoff depends on both the vector and the rank k. Computing it during every query is expensive, while storing every cutoff creates a dense object-by-rank table. As collections grow, both the routing cost and the cutoff cost become the bottleneck.

High-dimensional RkNN methods built on proximity graphs work in two stages: candidate generation avoids a full scan, and verification then tests the reverse predicate for each candidate. Fig. 2(A) shows why both stages are difficult. For a base vector o, let $r_k(o)$ be the distance to its kth nearest other base vector. In Euclidean (L_2) space,

$$o \in Rk\mathrm{NN}(q) \iff \delta(q,o)^2 \le r_k(o)^2. \tag{1}$$

At $k = 6$, the nearby vector A lies in a dense region and rejects q because its acceptance radius is small, whereas the farther vector B lies in a sparser region and accepts q. Base-space distance therefore ranks a rejector ahead of an acceptor, which

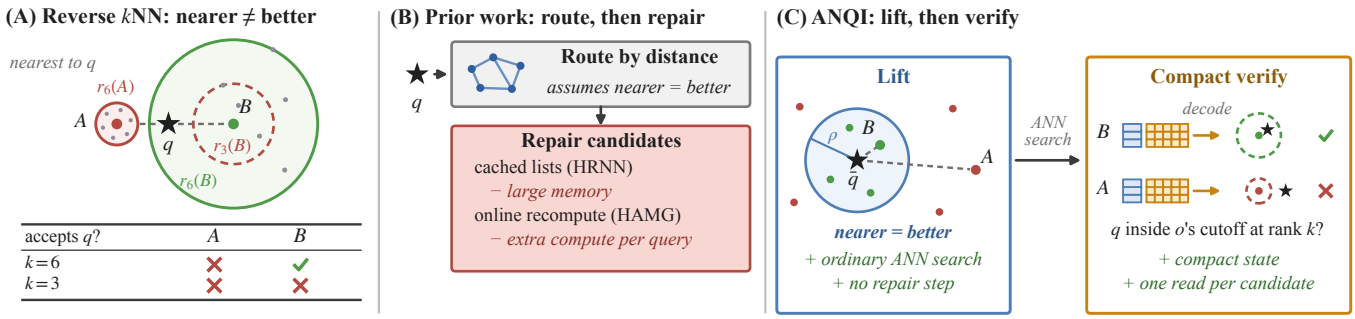


Fig. 2. Why routing must change. (A) In reverse kNN, nearer is not better: A is nearest but rejects q, while the farther B accepts it at $k = 6$. (B) Prior systems route by distance, then repair each candidate. (C) ANQI lifts each cutoff into routing, so nearer is better again, then verifies compactly.

misleads candidate generation. Verification faces a second difficulty: when the request changes to $k = 3$, B also rejects q, so a different cutoff must be recovered for each rank.

Fig. 2(B) shows how existing high-dimensional systems respond: they route through an ordinary proximity structure first, then repair the result for each candidate's acceptance cutoff. The repair is a second cost, and where it is paid determines how a system behaves as the collection grows (Section II): HRNN [4] caches it and already exceeds common server memory by one hundred million vectors when its own index is extrapolated (Section V), whereas HAMG [5] estimates it online through a multi-hop walk tested only up to ten million vectors.

ANQI avoids both costs by removing the repair step instead of paying for it. Fig. 2(C) shows the design: rather than routing a query through ordinary proximity and fixing the candidates afterward, ANQI folds each base vector's acceptance cutoff into the coordinate used for routing itself, so an ordinary forward search already favors vectors that are likely to accept q, using the cutoff estimates and candidate IDs from one pass of approximate kNN (AKNN) search (Section III). A standard norm-equalization step then turns the reverse-aware score into Euclidean proximity, so mature ANN graphs such as HNSW [6], NSG [7], or DiskANN [8] can retrieve reverse candidates. The lifted graph supplies candidate coverage, while an independent target-k verifier makes the final decision from a low-rank-plus-residual representation of the object-by-rank cutoff matrix (Section IV-B) without storing a full-precision cutoff table.

Table I compares direct high-dimensional reverse-neighbor indexes, where Scale is the largest collection evaluated in each cited work (for ANQI, see Section V-D). As the table shows, ANQI handles both Euclidean distance (L_2) and inner product (IP) with one graph-and-verifier architecture and extends the largest scale reported for direct indexes from ten million to one billion vectors. One index also serves every rank materialized at build time. ANQI makes three main contributions:

- We cast RkNN membership under both metrics as one threshold test on an inner-product score and place each base vector's reverse decision boundary directly in the indexed score, allowing graph search to reach reverse neighbors with less work.

TABLE I
DIRECT REVERSE-NEIGHBOR INDEXES: MECHANISM, CAPABILITIES, AND SCALE.

Method	Mechanism	L_2	IP	Approx.	Scale
SFT [9]	Tree; forward kNN of q, range-query test	✓	–	✓	10^4
RDT [10]	Tree; dimensional tests, exact kNN refinement	✓	–	✓	10^6
HAMG [5]	Graph; multi-hop walk, online radius estimate	✓	–	✓	10^7
HRNN [4]	Graph; stored ranked and reverse lists	✓	–	✓	10^6
SIMPFER [1]	Bounds; per-user score bounds, exact	–	✓	–	10^6
SAH [11]	Hashing; shifted transform, cone tree	–	✓	✓	10^6
ANQI	Graph; lifted routing, compact verifier	✓	✓	✓	$\mathbf{10^9}$

- We model vector- and rank-specific cutoffs with a low-rank representation plus learned residual quantization (LRQ), storing a few coefficients and one packed code per rank for each object. For L_2, the Local Radius Law motivates the shared rank structure; for IP, the same representation is fitted directly to the AKNN cutoffs.
- We evaluate the integrated system on five million-scale datasets under both predicates. At high-recall operating points (near 0.95 for L_2 and 0.99 for IP), ANQI is 2.51–2050× faster than the fastest baseline at the same recall, with precision between 0.97 and 1.00, although its build, which includes cutoff generation, costs more than plain HNSW or the IP baselines (Table VI). Because the boundary is folded into routing geometry rather than repaired afterward, ANQI is also not bound by HRNN's storage wall: on SIFT1B at $k = 10$ it completes index construction and answers billion-vector queries end to end with recall .98, precision .97, and 17 k QPS—to our knowledge the first such reverse-kNN demonstration.

II. Related Work

ANQI draws on five lines of work: proximity graphs for candidate generation, reverse-neighbor indexes, dimension lifting for reverse-aware scoring, maximum-inner-product search (MIPS) graphs for the inner-product case, and neighbor-distance models for the compact verifier.

a) Proximity graph indexes: High-dimensional ANN search spans locality-sensitive hashing [12]–[14], adaptive tree indexes [15], and quantized billion-scale engines [16], but proximity graphs route a query through sparse data-dependent neighborhoods instead. Early graph search hill-climbs over an offline-built kNN graph [17]; NN-Descent [18] makes that graph itself scalable to build, and EFANNA [19] improves the starting point for both construction and search with random-projection-tree seeding. Pruning then splits into two distinct families. FANNG [20], HNSW [6], and NSG [7] drop an edge once a closer kept neighbor already dominates it (occlusion pruning); PANNG [21] instead drops an edge whenever a short alternate path survives, and ONNG [22] adds a minimum-in-degree guarantee on top of PANNG to repair the reachability that path pruning alone destroys. Recent systems push along orthogonal axes: our own prior work, Vista [23], gives low-in-degree (anti-hub) vectors a larger, connectivity-aware edge budget during construction rather than one fixed for every vector; DiskANN/Vamana [8] targets billion-scale single-node construction; ParlayANN [24] gives several of these algorithms a deterministic, lock-free parallel implementation rather than a new topology; and graph-based range retrieval [25] replaces top-k output with adaptive early stopping for a query radius, evaluated to one billion vectors. ANQI's lift (Section IV-A) turns reverse-neighbor candidate generation into exactly this forward-ANN problem, so advances along any of these axes apply to ANQI's graph unchanged, with the reverse-aware geometry as the new part.

b) Reverse-kNN indexes: Reverse-neighbor search began as an influence query and later gained exact spatial and metric indexes [3], [26], [27]. Methods for high-dimensional data commonly separate candidate generation from vector-specific verification. Tree-based systems start from forward neighbors (SFT [9]) or combine local tests with forward-kNN refinement (RDT [10]). Graph systems instead broaden candidate coverage in the base space. HAMG [5] uses multi-hop expansion with an online radius recomputation untested beyond ten million vectors, while HRNN [4] uses materialized reverse lists, so its measured ten-million-vector index already occupies $86.7\,\mathrm{GB}$, which exceeds common server memory by one hundred million vectors when extrapolated. Of these, HRNN is our closest direct competitor. For inner product, reverse-MIPS methods specialize the predicate: SIMPFER [1], [2] prunes candidates with per-user score bounds. Concurrent RT-RKNN [28] solves an exact two-dimensional spatial variant with GPU ray tracing, a setting that differs from high-dimensional vector search. ANQI retains the same candidate-generation/verification decomposition as these systems, but builds the acceptance boundary into the routing coordinate itself rather than restoring it afterward, and uses one such architecture for both L_2 and inner product. Table I compares their mechanisms side by side.

c) Dimension lifting: The L_2 reverse predicate is a point-enclosure query over balls $B(o, r_k(o))$. Its signed power distance, $\|q-o\|^2 - r_k(o)^2$, belongs to the classical geometry of power diagrams and sphere-to-halfspace lifting [29]. Cheong et al. [30] use related geometry for exact reverse-neighbor search in fixed dimension. Arthur–Oudot [31] construct lifted LSH subproblems for high-dimensional reverse 1-NN.

ANQI builds on this geometric lineage but targets a different high-dimensional index interface. It derives the scalar correction from a rank-dependent local-neighbor cutoff, builds one approximate graph for candidate generation, and applies compact target-k verification. The closest transform-based reverse-MIPS design, SAH [11], combines a shifting-aware transform with hashing and cone-tree search.

d) Graph indexes for MIPS: After lifting, inner-product reverse search becomes a forward MIPS problem, so ANQI also builds on graph indexes for MIPS. MIPS first motivated asymmetric hashing reductions [32]–[35]; FARGO [36] refines this line with multi-probing across several hash tables rather than a graph. Graph-based methods instead split over whether they transform the predicate or solve it natively. IP-NSW [37] argues against any reduction and builds directly in inner-product space, but gives disproportionate in-degree to large-norm vectors (norm bias); IP-NSW+ [38] corrects this at traversal time with an auxiliary angular graph that seeds the search, while NAPG [39] corrects the same bias at construction time by learning a norm-adjustment factor for edge selection, since inner product has no triangle inequality to prune by directly. The alternative line reduces MIPS to an already-solvable graph problem instead: the Möbius transformation [40] applies one fixed geometric map between L_2- and IP-Delaunay graphs, while PSP [41] reduces MIPS to a query-scaled nearest-neighbor problem so an ordinary graph index and pruning rule apply unchanged. MAG [42] argues both lines have complementary failure modes (native graphs stall in local optima, transformed graphs distort topology) and fuses both metrics at index and search time. None of these methods represent a base vector's own rank-dependent reverse-acceptance boundary; ANQI adds exactly that on top of this forward-MIPS graph machinery.

e) Neighbor-distance modeling and compression: Prior work models vector-specific k-distance curves to reduce storage or support a query-time rank. Achtert et al. fit a log–log power model [43]. Berrendorf et al. learn compact bounds with safe refinement [44]. Obermeier et al. use nonlinear approximation [45], and HAMG [5] extrapolates local distances during pruning. These results motivate compact modeling of rank-dependent neighbor thresholds. Low-rank approximation and vector quantization provide complementary compression tools [46]–[53]. ANQI applies these principles to the vector-by-rank matrix of scalar thresholds: its verifier combines a low-rank model with learned packed residual quantization, which avoids a dense full-precision threshold table.

III. Problem and Overview

Before ANQI's technical details, we present an overview on the reverse k-NN problem, our key ideas and design goals. This section formalizes the problem (Section III-A), states ANQI's design goals and key ideas (Section III-B), and walks through its end-to-end workflow (Fig. 3, Section III-C).

A. Problem Formulation

We fix notation and state the problem. Throughout, let $\mathcal{X} = \{o_1, \ldots, o_n\} \subset \mathbb{R}^D$ be a collection of n base vectors. We use $\langle \cdot, \cdot \rangle$ for inner product, $\|\cdot\|$ for Euclidean norm, and $\delta(x, y) = \|x - y\|$ for Euclidean distance. The definitions and algorithms in this paper target Euclidean distance and inner product.

Definition 1 (Reverse k-NN). For a query $q \in \mathbb{R}^D$ and requested rank k, let $r_k(o)$ be the distance from $o \in \mathcal{X}$ to its kth nearest vector in $\mathcal{X} \setminus \{o\}$, and let $\mathrm{ip}_k(o)$ be its kth largest inner product with a vector in $\mathcal{X} \setminus \{o\}$. Threshold ties are included. The reverse-kNN answer $A_k(q)$ contains every base vector for which q is among the k best matches. Under L_2 distance,

$$A_k(q) = R k\mathrm{NN}(q) = \{o \in \mathcal{X} : \delta(q, o)^2 \le r_k(o)^2\}. \quad (2)$$

Under inner product,

$$A_k(q) = R k\mathrm{NN}(q) = \{o \in \mathcal{X} : \langle q, o \rangle \ge \mathrm{ip}_k(o)\}. \quad (3)$$

The answer size is data-dependent and need not equal k.

a) One predicate for both metrics: The two definitions differ only in the direction of their native scores. We orient both as a smaller-is-better threshold test so that both metrics share one predicate and query interface:

$$\big(\kappa(q, o), t_k(o)\big) = \begin{cases} (\delta(q, o)^2, r_k(o)^2), & L_2, \\ (-\langle q, o \rangle, -\mathrm{ip}_k(o)), & \text{inner product.} \end{cases} \quad (4)$$

Both metrics therefore use the shared test $\kappa(q, o) \le t_k(o)$ for membership in $A_k(q)$, and we call $t_k(o)$ the cutoff (or threshold) of o at rank k.

b) Approximate indexed query: Evaluating this predicate for every base vector requires a full scan. An approximate index avoids this cost with two stages. Given (q, k), candidate generation retrieves a small set $C(q) \subseteq \mathcal{X}$ that should contain the true reverse neighbors. Verification then tests only these candidates. It uses a represented cutoff $\hat{t}_k(o)$ and returns

$$\widehat{R}_k(q) = \{o \in C(q) : \kappa(q, o) \le \hat{t}_k(o)\}. \quad (5)$$

This decomposition creates two independent error paths. First, a true reverse neighbor that is missing from $C(q)$ cannot be recovered by verification, so this error lowers recall. Second, an inaccurate $\hat{t}_k(o)$ can reject a true answer or accept a false one, so this error affects recall or precision. The returned set is exact only when $C(q)$ covers $A_k(q)$ and $\hat{t}_k(o) = t_k(o)$ for every candidate. We therefore measure the two errors separately.

Table II lists the principal notation. Tildes denote AKNN estimates, and hats denote verifier reconstructions.

c) Running example: One running example threads through the method: the toy of Fig. 2(A) with illustrative numbers, L_2 distance, and q at the origin. Vector A lies at distance 1 in a dense region ($r_6(A) = 0.5$, $r_3(A) = 0.3$), and B lies at distance 2 in a sparser one ($r_6(B) = 2.5$, $r_3(B) = 1.5$). At $k = 6$ only B accepts q; at $k = 3$ neither does. The graph is built at construction rank $K_{\mathrm{pivot}} = 6$ and then asked for both $k = 6$ and $k = 3$. The Base AKNN stage of Fig. 3 estimates the cutoffs and candidate-neighbor IDs in one pass.

TABLE II
PRINCIPAL NOTATION.

Symbol	Meaning
$\mathcal{X}, n, D$	Base-vector collection, cardinality, and dimension.
$q, o, k, \mathcal{K}$	Query, base vector, target rank, and represented rank set.
$r_k(o), \mathrm{ip}_k(o)$	Distance to o's kth nearest other base vector, and o's kth largest inner product with another base vector.
$\kappa(q, o), t_k(o)$	Oriented query score and true rank-specific cutoff.
$s_k(q, o)$	Lifted reverse-aware score used for candidate routing (eq. (9)).
$\widetilde{t}_k(o), \hat{t}_k(o)$	AKNN-derived cutoff and LRQ-reconstructed cutoff.
$A_{k(q)}, C(q), \widehat{R}_{k(q)}$	Exact answer, candidate set, and returned answer.
$T, \widetilde{T}, M, b$	True and AKNN cutoff matrices, model rank, and residual width.
K_{pivot}, G, L	Construction rank, graph degree, and descent width.
$\mathcal{G}, \mathcal{E}, \mathcal{V}, \mathcal{I}$	Graph, entry structure, verifier state, and index.
$\Pi, \theta_k(o)$	Query profile, and decoded cutoff after its verifier slack.

B. Design Goals and Key Ideas

ANQI pursues three goals, one for each cost of Section I and one for scale. Each is met by one key idea and tested by a controlled experiment in Section V.

- **Route by the reverse predicate.** A standard forward-ANN graph should reach reverse neighbors without expanding many extra candidates. ANQI lifts each base vector with its own cutoff, so reverse membership becomes a Euclidean ball test around the query under either metric (Section IV-A). At equal recall, lifted routing needs up to $3.5\times$ less graph work than routing by distance alone (Section V-F, Panels G–H of Fig. 5).
- **Verify from compact state.** Each candidate should be decided at the requested rank without recomputing cutoffs or storing a full-precision vector-by-rank table. ANQI models the cutoff matrix with a shared low-rank basis and a learned residual code (LRQ), so one scalar read per candidate suffices (Sections IV-B and IV-C). The verifier state takes 132 bytes per vector, a third of the dense matrix's 400, and recall stays at or above 0.994 on all five datasets (Section V-F, Panels C–F).
- **Serve every rank from one build.** One index should answer every rank materialized at build time and still build at billion scale. ANQI takes neighbor IDs and cutoffs from a single AKNN pass and builds one graph at a construction rank. The answer set of a smaller rank is contained in that of a larger rank, and a score correction swaps in the requested rank's cutoff at query time (Section IV-D). Across construction ranks 25–200, one graph reaches recall 0.99 at $k = 10, 50, 100$, and the design scales to one billion vectors (Sections V-D and V-F).

The first two goals give ANQI its two pieces, a lifted graph that generates candidates and a compact verifier that decides. They keep separate state and meet only at verification.

C. Workflow Overview

Fig. 3 summarizes ANQI's construction and query workflows, showing how an AKNN-derived threshold source supports both reverse-aware candidate generation and compact candidate verification. During construction, a single base-space AKNN pass produces an approximate neighbor table $\mathcal{W}_{\mathrm{base}}$ and cutoff matrix $\widetilde{T}$, which are reused to initialize graph construction and

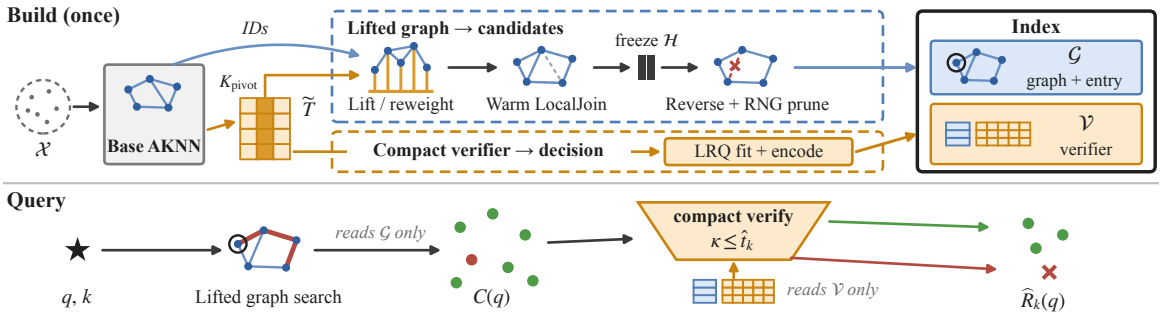


Fig. 3. ANQI workflow. One AKNN pass feeds two independent pieces: a lifted graph that generates candidates (blue, Section IV-A) and a compact verifier that decides (orange, Section IV-B). They meet at verification.

train the verifier without a separate threshold-generation pass. The blue path uses the neighbor IDs and construction-rank column $\widetilde{T}[:, K_{\text{pivot}}]$ to lift the vectors, equalize their norms, and reweight the warm-start neighborhoods before LocalJoin refinement, reverse gathering, and RNG pruning produce the bounded-degree graph $\mathcal{G}$. This embeds each reverse cutoff in the routing geometry, so traversal favors likely reverse neighbors over merely nearby objects. Meanwhile, the orange path fits LRQ to $\widetilde{T}$ and stores shared low-rank factors plus quantized residuals in $\mathcal{V}$, reconstructing target-rank cutoffs without a dense full-precision table. At query time, ANQI decodes the requested cutoff column once per batch, searches $\mathcal{G}$ to form $C(q)$, expands the target-rank envelope when needed, and reuses recorded scores for verification, leaving $\mathcal{G}$ responsible for candidate coverage and $\mathcal{V}$ for boundary fidelity.

a) Index construction: Algorithm 1 lists the build row of Fig. 3. After the AKNN pass, ANQI fits the verifier (FitAndEncodeLRQ, Section IV-B) and lifts every vector with its construction-rank cutoff (ReverseLift and EqualizeNorms, Section IV-A). Because lifted-space neighbors differ from base-space ones, ANQI reweights the AKNN IDs in lifted space instead of running a second AKNN build, refines them with warm LocalJoin [18] for at most J rounds, and finally gathers reverse neighbors and prunes them to degree G. The index then persists the graph, the entry structure that supplies query seeds, each vector's construction-rank cutoff, and the verifier's basis, coefficients, codes, and codebooks. Base vectors stay shared, and the AKNN tables, lifted coordinates, and construction pools are discarded. Queue handling, entry seeds, and the other implementation details of both algorithms are in Appendix A-L.

b) Query processing: Algorithm 2 follows the query row of Fig. 3. For each batch, ANQI first decodes the requested rank's cutoff column once, relaxed by the verifier slack of the query profile Π (zero by default). Each query then enters the graph at the entry seeds that Π specifies and scores every visited vector in base space. That score combines with the vector's stored construction-rank cutoff into a route key, and a best-first descent of width L follows the smallest keys. The collection envelope of Π, adjusted by the decoded cutoff, decides which visited vectors are collected as candidates, and

Algorithm 1 ANQI index construction.

Require: base vectors $\mathcal{X}$, ranks $\mathcal{K}$, K_{pivot}, degree G, metric m, LRQ parameters M, b, refinement limit J

procedure BuildIndex($\mathcal{X}, \mathcal{K}, K_{\text{pivot}}, G, \mathsf{m}, M, b, J$)
  $(\mathcal{W}_{\text{base}}, \widetilde{T}) \leftarrow$ ApproxKNN($\mathcal{X}, \max \mathcal{K}, \mathsf{m}$) ▷ IDs and cutoff estimates
  $\mathcal{V} \leftarrow$ FitAndEncodeLRQ($\widetilde{T}, M, b$)
  $\{z_o\} \leftarrow \{$ReverseLift$(o, \widetilde{T}[o, K_{\text{pivot}}], \mathsf{m}) : o \in \mathcal{X}\}$
  $\{\bar{o}\} \leftarrow$ EqualizeNorms$(\{z_o\})$
  $\mathcal{H} \leftarrow$ ReweightWarmStart$(\mathcal{W}_{\text{base}}, \{\bar{o}\})$
  for $j = 1, \ldots, J$ **do**
    $(\mathcal{H}, \Delta_j) \leftarrow$ LocalJoin$(\mathcal{H}, \{\bar{o}\})$
    if $\Delta_j < n/200$ **then**
      break
  $\mathcal{H}^\star \leftarrow$ Freeze$(\mathcal{H})$
  $\mathcal{P} \leftarrow$ ReverseAndGather$(\mathcal{H}^\star, \{\bar{o}\})$
  $\mathcal{G} \leftarrow$ DestinationRNGPrune$(\mathcal{P}, G)$
  $\mathcal{E} \leftarrow$ BuildEntryStructure$(\{\bar{o}\})$
  $\mathcal{I} \leftarrow$ PersistCompact$(\mathcal{G}, \mathcal{E}, (\widetilde{t}_{K_{\text{pivot}}}(o))_{o \in \mathcal{X}}, \mathcal{V}, K_{\text{pivot}}, \mathsf{m})$
  DiscardTemporary$(\mathcal{W}_{\text{base}}, \widetilde{T}, \{z_o\}, \{\bar{o}\}, \mathcal{H}, \mathcal{P})$
  return $\mathcal{I}$

a second phase floods outward from them without an L cap. The final comparison reuses the score recorded at discovery. Search returns a variable-size answer, and L controls traversal effort rather than answer cardinality.

IV. Lifting and Compact Verification

We detail the lift (Section IV-A), compact verifier (Sections IV-B and IV-C), and one-graph rank reuse (Section IV-D). See Appendix A for proofs.

A. Lifting for Candidate Generation

The lift lets a forward-kNN graph such as HNSW [6], NSG [7], or DiskANN [8] route by the reverse predicate. Such graphs order vectors by native distance or similarity to q, whereas reverse membership depends on a different cutoff for every base vector. ANQI therefore makes that cutoff part of the indexed vector, so that vectors likely to accept q are visited early by an ordinary forward traversal. For fixed k, expanding the L_2 predicate gives

$$\begin{aligned} \|q - o\|^2 \le r_k(o)^2 &\iff \|q\|^2 + \|o\|^2 - 2 \cdot \langle q, o \rangle \le r_k(o)^2 \\ &\iff \langle q, o \rangle - \tfrac{1}{2} \cdot \left(\|o\|^2 - r_k(o)^2\right) \ge \tfrac{1}{2} \cdot \|q\|^2. \end{aligned} \tag{6}$$

The left side combines the query–object interaction with the acceptance radius of o, while the right side is identical for every object considered by the query. The varying radii can

Algorithm 2 ANQI batched reverse-kNN search.

Require: compact index $\mathcal{I}$, shared base $\mathcal{X}$, batch Q, rank k, width L, query profile Π

procedure SEARCHBATCH($\mathcal{I}, \mathcal{X}, Q, k, L, \Pi$)
  $(\hat{t}_k(o), \theta_k(o))_{o\in\mathcal{X}} \leftarrow$ DECODECUTOFFS($\mathcal{V}, k, \Pi$) ▷ transient per batch
  for all $q \in Q$ **in parallel do**
    $S \leftarrow$ ENTRYSEEDS($\mathcal{E}, q, \Pi$)
    initialize bounded $\mathcal{Q}_b$, FIFO $\mathcal{Q}_f$, $C(q)$, score, and seen
    for all $e \in S$ **do**
      EXPLORE(e, true)
    while $\mathcal{Q}_b$ has an unexpanded item **do**
      $u \leftarrow$ CLOSESTUNEXPANDED($\mathcal{Q}_b$)
      for all $v \in N_{\mathcal{G}}(u)$ **do**
        EXPLORE(v, true)
    while $\mathcal{Q}_f$ is not empty **do**
      $u \leftarrow$ POP($\mathcal{Q}_f$)
      for all $v \in N_{\mathcal{G}}(u)$ **do**
        EXPLORE(v, false)
    $\widehat{R}_k(q) \leftarrow \{o \in C(q) : \mathsf{score}[o] \le \theta_k(o)\}$
  return $\{\widehat{R}_k(q) : q \in Q\}$
function EXPLORE(v, descend)
  if v is unseen **then**
    mark v, set $\kappa_v \leftarrow$ BASESCORE($q, \mathcal{X}[v]$, m) and $\mathsf{score}[v] \leftarrow \kappa_v$
    $d_v \leftarrow$ LIFTEDROUTEKEY($\kappa_v, \widetilde{t}_{K_{\mathrm{pivot}}}(v)$, m)
    if descend **then**
      INSERTBOUNDED($\mathcal{Q}_b, (d_v, v), L$)
    if INFLOODENVELOPE($v, \kappa_v, d_v, k, \mathcal{I}, \Pi$) **then**
      PUSH($\mathcal{Q}_f, v$)
    if INCANDIDATEENVELOPE($v, \kappa_v, d_v, k, \mathcal{I}, \Pi$) **then**
      $C(q) \leftarrow C(q) \cup \{v\}$

therefore influence candidate order instead of being consulted only after candidate discovery. Inner product admits the same separation:

$$\langle q, o\rangle \ge \mathrm{ip}_k(o) \iff \langle q, o\rangle - \mathrm{ip}_k(o) \ge 0. \tag{7}$$

In both metrics, the reverse predicate compares an object-side score with a query-only cutoff. The score is the native query–object interaction plus a scalar correction known at construction time. ANQI appends that correction to the object as one extra coordinate and appends a constant 1 to the query, giving the lifted vectors (superscript ℓ):

$$o_k^\ell = \begin{cases} [o, -\frac{1}{2}\cdot(\|o\|^2 - r_k(o)^2)], & L_2, \\ [o, -\mathrm{ip}_k(o)], & \text{inner product}, \end{cases} \qquad q^\ell = [q, 1]. \tag{8}$$

Let $s_k(q, o) = \langle q^\ell, o_k^\ell\rangle$. Equations (6) and (7) then give

$$o \in A_k(q) \iff s_k(q, o) \ge \tau(q), \qquad \tau(q) = \begin{cases} \frac{1}{2}\cdot\|q\|^2, & L_2, \\ 0, & \text{inner product}. \end{cases} \tag{9}$$

The lifted inner product reproduces the reverse-aware score without an object-specific query-time operation. The vector-specific threshold is part of the indexed score, and reverse membership becomes a forward maximum-inner-product test.

Euclidean ANN graphs expect a distance, so we use the standard XBOX reduction [54]: one more coordinate pads every lifted vector to the same norm, which makes inner-product order and Euclidean order coincide. The reverse-specific step is the preceding threshold lift, which places each object's rank-dependent acceptance cutoff in the routing geometry. Choose $M_\star \ge \max_{o\in\mathcal{X}}\|o_k^\ell\|$ and define

$$\bar{o}_k = \left[o_k^\ell, \sqrt{M_\star^2 - \|o_k^\ell\|^2}\right], \qquad \bar{q} = [q^\ell, 0]. \tag{10}$$

Every $\bar{o}_k$ has norm $M_\star$, and

$$\|\bar{q} - \bar{o}_k\|^2 = \|q^\ell\|^2 + M_\star^2 - 2\cdot s_k(q, o). \tag{11}$$

A larger reverse-aware score is therefore exactly equivalent to a shorter Euclidean distance. The membership test becomes

$$o \in A_k(q) \iff \|\bar{q} - \bar{o}_k\|^2 \le \underbrace{\|q^\ell\|^2 + M_\star^2 - 2\cdot\tau(q)}_{\rho_k(q)^2}. \tag{12}$$

Theorem 1 (Exact reverse-aware candidate geometry)**.** *Assume exact target-k thresholds. For either L_2 or inner product, the construction above has three properties. First, the lifted ball test in* (12) *is equivalent to exact reverse membership. Second, ordering base vectors by increasing $\|\bar{q} - \bar{o}_k\|$ is equivalent to ordering them by decreasing reverse-aware score $s_k(q, o)$. Third, filtering any candidate set $C(q)$ in the lifted space returns exactly $C(q) \cap A_k(q)$.*

See Appendix A-A for the proof, which substitutes the lifted score into the equal-distance identity.

The lift thus replaces many vector-centered acceptance balls with one query-centered ordering that a forward ANN graph can navigate (Fig. 2(C)). The theorem assumes exact thresholds t_k, whereas deployment substitutes the AKNN estimates $\widetilde{t}_k$, so candidate coverage is measured end to end. Returning to the example, the rank-6 lift gives $s_6(q, B) = -\frac{1}{2}(4 - 6.25) = 1.125$ and $s_6(q, A) = -\frac{1}{2}(1 - 0.25) = -0.375$ against $\tau(q) = 0$, so a forward search for $\bar{q}$ reaches B before A although A is nearer, with nothing cached in advance and nothing recomputed from scratch. Whether B is returned at a given k is decided next, by the verifier.

B. Compact Threshold Modeling

Verification needs one object-specific threshold at the requested rank, but storing a full-precision scalar for every object and rank creates a dense matrix. ANQI exploits the structure shared across rank columns and reserves a small residual code for the detail that the shared model does not capture.

a) Euclidean structure: For Euclidean data, the threshold is $t_k(o) = r_k(o)^2$, and its dependence on k has a simple local form. Around an object, points are spread roughly evenly, so the mass inside a ball grows with its volume, as in classical kNN density estimation and nearest-neighbor limit theory [55]–[57]. In a neighborhood where sampling intensity and intrinsic dimension [58], [59] are approximately constant, write $N_o(r) = \rho(o)\cdot V_{d(o)}\cdot r^{d(o)}$. Here $N_o(r)$ is the expected number of neighbors within radius r, $\rho(o)$ the local intensity, $d(o)$ the local intrinsic dimension, and V_d the unit-ball volume. Setting $N_o(r_k(o)) = k$ gives a rank–radius relation that is algebraic rather than a fitted curve.

Proposition 1 (Euclidean Local Radius Law)**.** *Under the local ball-mass model,*

$$r_k(o)^2 = \gamma(o) \cdot k^{\nu(o)}, \qquad \nu(o) = \frac{2}{d(o)}, \quad \gamma(o) = \left[\rho(o) \cdot V_{d(o)}\right]^{-\nu(o)}. \tag{13}$$

In log–log coordinates the law is a straight line,

$$\log r_k(o) = \frac{1}{d(o)} \left[\log k - \log\big(\rho(o) \cdot V_{d(o)}\big)\right],$$

whose slope is the inverse local dimension and whose intercept reflects local density. Different objects have different slopes and intercepts, so one global radius schedule is insufficient. Writing $\nu(o) = \nu_0 + \beta(o)$ around a shared exponent ν_0 and expanding the exponential yields the separable form

$$r_k(o)^2 = \sum_{m=0}^{\infty} \underbrace{\frac{\gamma(o) \cdot \beta(o)^m}{m!}}_{f_m(o)} \underbrace{k^{\nu_0} \cdot (\ln k)^m}_{\phi_m(k)}. \tag{14}$$

Every term multiplies an object-specific factor $f_m(o)$ by a rank-specific factor $\phi_m(k)$, so the object-by-rank threshold matrix is approximately low-rank: a few shared rank functions suffice over a rank interval where $|\beta(o) \cdot \ln k|$ is moderate, even when the ambient dimension D is large. The assumption is local in scale: curvature, changing density, or a changing intrinsic dimension may alter the power law over a wider interval. The truncation error and the complete derivation are in Appendix A-B.

This analysis suggests the layout that ANQI stores. For the AKNN-derived vector-by-rank matrix $\widetilde{T}[o,k] = \widetilde{t}_k(o)$ used by the deployed index, we write

$$\widetilde{t}_k(o) = \mu_k + \sum_{m=1}^{M} c_m(o) \cdot g_{m,k} + e_{o,k}. \tag{15}$$

The mean μ_k captures the collection-wide rank trend, and the shared basis $g_{m,k}$ the regular rank variation suggested above. The coefficients $c_m(o)$ modulate that basis according to each object's neighborhood structure, and the residual $e_{o,k}$ records the departures that a finite basis cannot reproduce. Each object stores M coefficients and a packed code for $e_{o,k}$, which we call learned residual quantization (LRQ). Before a query batch, ANQI decodes only the requested threshold column. For each rank, the ordered codebook $\mathcal{B}_k$ includes a lower endpoint no larger than every encoded residual, and the raw floor decoder rounds each residual down to a codebook value:

$$\begin{aligned} \tilde{e}_{o,k} &= \max\{b \in \mathcal{B}_k : b \le e_{o,k}\}, \\ \hat{t}_k(o) &= \mu_k + \sum_{m=1}^{M} c_m(o) \cdot g_{m,k} + \tilde{e}_{o,k}. \end{aligned} \tag{16}$$

Verification then needs one scalar read per candidate. The deployed configuration uses $M = 8$ and learned 8-bit (u8) residual quantization. A standalone control sets $e_{o,k} = 0$ and fits an empirical M-only basis, isolating the rank structure the shared term captures before residual correction (Section V-E).

b) Inner-product thresholds: Inner product has no radius-volume relation. Its thresholds instead form directional score quantiles. The following identity characterizes the true IP cutoff. ANQI applies the same low-rank-plus-LRQ layout to its AKNN estimate.

Proposition 2 (Projection quantiles for inner product)**.** *For a nonzero object o, let $u_o = o/\|o\|$, and let $\pi_k(o)$ be the kth largest value in $\{\langle x, u_o\rangle : x \in \mathcal{X} \setminus \{o\}\}$. Then*

$$\mathrm{ip}_k(o) = \|o\| \cdot \pi_k(o), \qquad o \in A_k(q) \iff \langle q, u_o\rangle \ge \pi_k(o). \tag{17}$$

Equivalently, the signed verifier row is $T[o,k] = -\|o\| \cdot \pi_k(o)$. If $o = 0$, the row is identically zero.

The proof factors the positive scalar $\|o\|$ from each inner product. See Appendix A-C.

Across ranks, each true signed IP row is therefore the negative upper-quantile curve of projections onto u_o: the object norm sets the row scale, and the projection order sets how the cutoff changes with rank. The deployed index estimates these rows with its base-space AKNN pass, $\widetilde{T}[o,k] = -\widetilde{\mathrm{ip}}_k(o)$. ANQI learns a shared basis from this matrix, and LRQ stores the remaining object- and rank-specific deviation. Thus L_2 and inner product share one stored layout and decoder for different structural reasons: local ball growth motivates the Euclidean basis, whereas the correlated directional quantiles of inner product are learned directly. In both cases, the query-time check keeps the same threshold-comparison form.

Returning to the example, reconstructing B's rank-3 cutoff needs only its M stored coefficients and its rank-3 residual code via eq. (16), with no full-precision entry of a dense $n \times |\mathcal{K}|$ table. It returns $\hat{t}_3(B) \approx 2.25$, below $\kappa(q,B) = 4$, so B is rejected at $k = 3$. At $k = 6$ the same coefficients with the rank-6 code return $\hat{t}_6(B) \approx 6.25 \ge 4$, so B is accepted, while A ($\kappa = 1$) is rejected at both ranks.

C. Verification Guarantees

Because the floor decoder in (16) only ever rounds a residual down, LRQ can make the verifier stricter than its own AKNN source but never more lenient: compression can cost recall, but it cannot add false positives beyond those the source already contains. Let $\hat{A}_k(q) = \{o : \kappa(q,o) \le \hat{t}_k(o)\}$ denote full-base verification under the represented cutoff $\hat{t}_k$.

Proposition 3 (One-sided threshold guarantees)**.** *Assume exact target thresholds. If $\hat{t}_k(o) \le t_k(o)$ for every o, then $\hat{A}_k(q) \subseteq A_k(q)$ and verification introduces no false positives. Its false negatives are confined to*

$$\{o : \hat{t}_k(o) < \kappa(q,o) \le t_k(o)\}. \tag{18}$$

If $\hat{t}_k(o) \ge t_k(o)$ for every o, then $A_k(q) \subseteq \hat{A}_k(q)$ and verification introduces no false negatives.

The proof chains the represented and exact inequalities with $\kappa(q,o)$. See Appendix A-D.

In deployment, the source cutoff $\widetilde{t}_k$ is itself an estimate, so source and compression errors affect both measured recall and precision.

For L_2, suppose queries follow the same local population as the base vectors and the total reconstructed cutoff is restrictive, falling short of $t_k(o)$ by at most a relative ϵ_k for every object o. Each object then accepts at least a $(1-\epsilon_k)^{d(o)/2}$ fraction of the queries it should accept, so the recall of verification alone, pooled over queries, satisfies $\mathrm{Recall}_{\mathrm{pool}} \geq (1-\epsilon_k)^{d_{\max}/2}$ with $d_{\max} = \max_o d(o)$. This bound concerns verifier retention, while graph coverage is measured separately. Its derivation and the effect of LRQ codebook spacing appear in Appendices A-E and A-F.

D. Graph Reuse Across Target Ranks

The verifier represents the cutoffs of a rank set $\mathcal{K}$, the ranks materialized at build time: $\mathcal{K} = \{1, \ldots, 100\}$ in our million-scale experiments and $\{10\}$ at billion scale. The graph, in contrast, is built once, at a construction rank K_{pivot}, with the AKNN threshold $\widetilde{t}_{K_{\mathrm{pivot}}}(o)$ in each lifted object. This construction rank is distinct from the target rank k requested by a query. Three results together justify serving every $k \in \mathcal{K}$ from this one graph: a smaller rank's true answers are always a subset of a larger rank's, so a graph generous enough for K_{pivot} already contains what every $k \leq K_{\mathrm{pivot}}$ needs (Proposition 4); the target-rank cutoff can be swapped in at query time without touching graph edges (Lemma 1); and the extra candidate shell that a rank beyond K_{pivot} requires can be quantified rather than assumed (Corollary 1).

Proposition 4 (Shared-rank candidate envelope)**.** *For exact thresholds in either base space and any $K \geq k$,*

$$A_k(q) \subseteq A_K(q). \tag{19}$$

Consequently, the exact lifted acceptance region at K is a candidate superset for every target rank no larger than K.

Neighbor radii are nondecreasing in k, whereas the kth inner-product order statistic is nonincreasing. The proof is in Appendix A-A.

For the L_2 ranks we evaluate beyond the construction rank, query-time collection substitutes the decoded target-k cutoff while retaining the graph edges.

Lemma 1 (Target-rank score correction)**.** *For any object-side cutoff $a(v)$, let $s_a(q,v) = \langle q, v\rangle - \frac{1}{2}\cdot(\|v\|^2 - a(v))$. For any two cutoffs a and b,*

$$s_a(q,v) + \tfrac{1}{2}\cdot\big(b(v) - a(v)\big) = s_b(q,v). \tag{20}$$

The deployed path sets $a = \widetilde{t}_{K_{\mathrm{pivot}}}$ and $b = \hat{t}_k$. Setting $a = t_{K_{\mathrm{pivot}}}$ and $b = t_k$ recovers the exact target-k predicate, including when $k > K_{\mathrm{pivot}}$.

Substitution cancels the construction-rank threshold, so a graph built at K_{pivot} can be queried at any other rank by adding half the cutoff difference to each score. The proof is in Appendix A-A. The deployed L_2 path therefore admits a candidate v when

$$s_{\widetilde{t}_{K_{\mathrm{pivot}}}}(q,v) + \tfrac{1}{2}\cdot\big(\hat{t}_k(v) - \widetilde{t}_{K_{\mathrm{pivot}}}(v)\big) \geq \tfrac{1}{2}\cdot\|q\|^2, \tag{21}$$

while the evaluated IP ranks satisfy $k \leq K_{\mathrm{pivot}}$, so the construction-rank envelope already covers them.

For Euclidean thresholds, the Local Radius Law also describes the added shell beyond the construction rank.

Corollary 1 (Local shell growth)**.** *Under the local ball-mass model, for $k \geq K$,*

$$\frac{r_k(o)^2}{r_K(o)^2} = \left(\frac{k}{K}\right)^{\nu(o)}, \qquad N_o(r_k(o)) - N_o(r_K(o)) = k - K. \tag{22}$$

Thus the additional local mass is $k-K$, while squared-radius inflation is governed by $\nu(o) = 2/d(o)$.

See Appendix A-B for the derivation. This result connects cross-rank candidate growth to local dimension.

Finally, a finite graph traversal must still reach the candidates that the corrected envelope admits. For a query with $|A_k(q)| > 0$, candidate set $C_k(q)$, and returned set $\widehat{R}_k(q) \subseteq C_k(q)$, end-to-end recall separates as

$$\mathrm{Recall}_q = \underbrace{\frac{|A_k(q)\cap C_k(q)|}{|A_k(q)|}}_{\text{candidate coverage}} \cdot \underbrace{\frac{|A_k(q)\cap \widehat{R}_k(q)|}{|A_k(q)\cap C_k(q)|}}_{\text{verifier retention}}. \tag{23}$$

The zero-denominator case and full derivation appear in Appendix A-G, and the conditional completeness result under target-envelope reachability in Appendix A-H. Panels I–K of Fig. 5 report the fixed graph's measured robustness.

V. Experimental Evaluation

In this section, we thouroughly evalaute ANQI's performance and design. We first measure complete quality–throughput frontiers as effectiveness and cost vary with the target rank k. And then comes the account for build time, index size, and online work before scaling to one billion vectors. Controlled experiments finally test the three goals of Section III-B one at a time: lifted routing, cutoff modeling, and graph reuse across target ranks with ablation studies.

A. Experimental Setup

In this benchmark, we cover evaluation of two distance metrics, several embedding types, and three target ranks. The L_2 datasets span learned embeddings, local image descriptors, and GIST vectors. The inner-product datasets contain recommendation and text embeddings. Each main-benchmark dataset has approximately one million base vectors, and every method is tested with held-out queries at $k \in \{10, 50, 100\}$. The scalability study is implemented on the SIFT dataset with scales of 10M, 100M, and 1B vectors. Table III summarizes the datasets and query workloads.

Baselines. For L_2 distance, SFT [9] filters a forward neighborhood, RDT [10] applies dimensional tests, HAMG [5] explores multiple hops, and HRNN [4] adds ranked and reverse lists to graph navigation. We use the public HRNN codebase and its HNSW baseline implementations. For IP distance, SIMPFER, H2-Simpfer, and SA-Simpfer [1] use score bounds

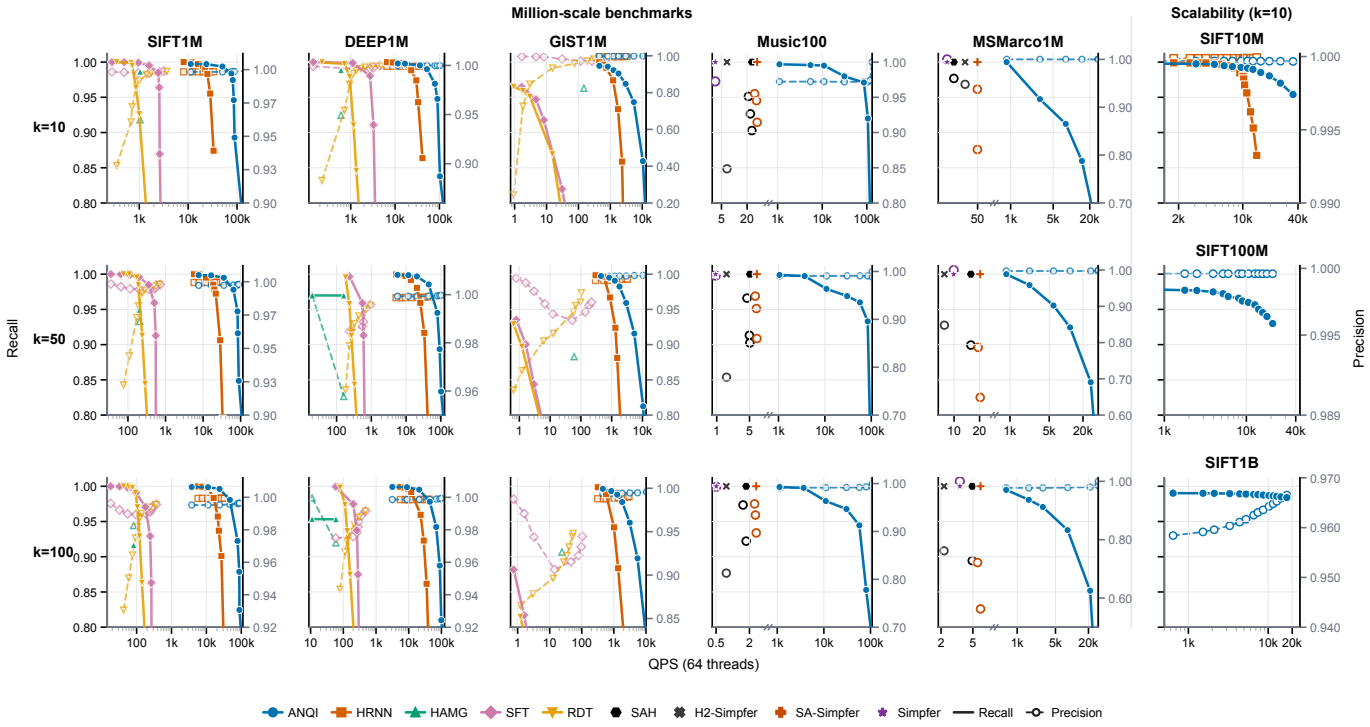


Fig. 4. End-to-end quality–throughput frontiers and SIFT scalability with Recall, Precision and QPS reported. The five left columns compare the million-scale datasets at $k = 10, 50, 100$. The separated right column reports reverse-10NN scalability from 10M to 1B vectors. Filled traces use the Recall axes, and hollow dashed traces use the Precision axes. Marked IP axis breaks omit empty QPS intervals.

and transformations. SAH/SA-Cone [11] adds shifting-aware hashing and a cone tree. We use the authors' repository.

Metrics. For a query collection $\mathcal{Q}$, we report retrieval quality against online and offline cost. Let $\mathcal{Q}_+ = \{q \in \mathcal{Q} : |A_k(q)| > 0\}$ and, for $q \in \mathcal{Q}_+$, write $A = A_k(q)$ and $\widehat{R} = \widehat{R}_k(q)$. Recall is $R_q = |A \cap \widehat{R}|/|A|$ and precision is $P_q = |A \cap \widehat{R}|/|\widehat{R}|$. MacroR and MacroP are their means over $\mathcal{Q}_+$, and F1 is computed per query and then averaged, so every query has equal weight regardless of answer-set size. If $\widehat{R}$ is empty, we set $P_q = 1$ because it contains no false positive; recall and F1 are then zero. Ground truth is exact: the answers on the canonical million-scale datasets come from exhaustive base-to-base computation, whereas ANQI's index uses AKNN-derived thresholds.

Protocol. We retain released defaults and tune only documented quality-cost controls. Following the baseline settings, the recall targets 0.95 for L_2 and 0.99 for industrial usage. All methods share queries and ground truth and use 64 workers on Intel Xeon Platinum 8457C sockets, with SMT disabled. IP implementations without internal batching distribute queries across workers while preserving their per-query routines.

TABLE III
DATASETS FOR THE MAIN BENCHMARK AND SCALABILITY STUDY.

Dataset	Source	Distance	n	Dim.	$\lvert Q\rvert$
SIFT1M	Image	L_2	1,000,000	128	10,000
DEEP1M	Image	L_2	1,000,000	96	10,000
GIST1M	Image	L_2	1,000,000	960	1,000
Music100	Music	IP	999,900	100	100
MSMarco1M	Text	IP	1,000,000	768	9,376
SIFT1B	Image	L_2	10^9	128	100,000

ANQI. Unless noted otherwise, our experiments use ANQI'sdefault evaluation implementation. Its default configuration uses an approximate AKNN top-100 result, a degree-48 graph for million-scale datasets and a degree-20 graph for 1B vectors at $K_{\text{pivot}} = 100$ with six LocalJoin passes, and an M8+u8 verifier shared by $k \in \{10, 50, 100\}$. It stores the lifted vectors and recomputes each candidate's base-space score during verification. The technical report and released code list every dataset-specific setting and search grid (Appendix A-J).

We also report QPS, build time, and index size, and a graoh-only count distance evaluation. Let $E_{\text{graph}}(q)$ count the graph-space full-vector distance evaluations made while traversing the index for query q, and report their average

$$\overline{E}_{\text{graph}} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} E_{\text{graph}}(q). \tag{24}$$

For ANQI, build time covers AKNN cutoff generation, LRQ fitting and encoding, lifting, graph construction, and persistence, and index size is the persisted state needed at query time, excluding the shared base vectors and construction temporaries.

TABLE IV
SELECTED HIGH-RECALL OPERATING POINTS.

L_2 Dataset	k	**ANQI**	HRNN [4]	HAMG [5]	SFT [9]	RDT [10]
SIFT1M	10	0.965/**0.998/84.3k/2.72k** [2.99×]	0.955/**0.998**/28.2k/6.62k	**0.986**/0.962/1.04k/180k	0.964/**0.998**/2.57k/161k	0.964/0.987/855/349k
	50	**0.964**/0.998/**74.6k/2.89k** [3.04×]	0.958/**0.999**/24.6k/7.81k	0.949‡/0.970/204/1.25M	0.959/0.995/504/1.29M	0.959/0.988/213/1.86M
	100	0.964/0.996/**60.9k/3.3k** [2.88×]	0.957/**0.999**/21.2k/8.52k	0.917‡/0.983/81.8/3.09M	**0.971**/0.990/179/4.11M	0.970/0.982/105/4.81M
DEEP1M	10	0.957/**1.000/88.3k/2.91k** [2.74×]	0.958/0.999/32.3k/6.85k	**0.989**/0.949/603/357k	0.981/0.997/2.73k/253k	0.979/0.981/946/470k
	50	0.958/**1.000/70.5k/3.41k** [2.75×]	0.959/0.999/25.6k/8.15k	**0.970**/0.958/161/1.89M	0.959/0.987/557/1.5M	0.958/0.979/233/2.07M
	100	0.954/**0.999/63.8k/3.75k** [2.76×]	0.960/**0.999**/23.1k/8.97k	0.953/0.972/59.3/5.13M	**0.975**/0.976/201/4.99M	0.974/0.966/112/5.49M
GIST1M	10	0.957/**0.998/4.29k/9.07k** [2.68×]	0.952/0.995/1.6k/15.3k	0.415‡/0.824/148/88.9k	**0.959**/0.997/2.37/17.8M	0.951/0.847/3.01/31M
	50	0.959/**0.997/2.95k/12.4k** [2.51×]	**0.960**/0.991/1.18k/21.8k	0.288‡/0.883/60.3/468k	0.935‡/0.994/0.80/54.1M	0.929‡/0.835/0.65/107M
	100	0.963/**0.994/2.53k/15.5k** [3.49×]	**0.980**/0.988/726/45.5k	0.241‡/0.927/24.0/1.43M	0.882‡/0.987/0.75/54.4M	0.815‡/0.865/1.21/47.2M
IP Dataset	k	**ANQI**	SAH [11]	H2-S [1]	SA-S [1]	SIMP. [1]
Music100	10	0.990/0.972/**21.4k** [622.3×]	**1.000**/0.903/25.8	**1.000**/0.849/6.74	**1.000**/0.915/34.5	**1.000/0.973**/3.58
	50	0.997/0.990/**5.38k** [748.8×]	**1.000**/0.851/5.08	**1.000**/0.779/1.62	**1.000**/0.860/7.18	**1.000/0.991**/0.95
	100	0.995/0.991/**5.42k** [2,046×]	**1.000**/0.880/1.74	**1.000**/0.813/0.76	**1.000**/0.897/2.65	**1.000/0.993**/0.50
MSMarco	10	**1.000/1.000/859** [17.23×]	**1.000**/0.960/32.1	**1.000**/0.948/39.6	**1.000**/0.812/49.9	**1.000/1.000**/28.2
	50	0.993/0.997/**1.18k** [57.98×]	**1.000**/0.793/15.8	**1.000**/0.848/7.68	**1.000**/0.649/20.4	**1.000/1.000**/9.90
	100	0.992/0.978/**862** [138.4×]	**1.000**/0.728/4.91	**1.000**/0.762/2.18	**1.000**/0.563/6.23	**1.000/1.000**/3.43

Recall targets are 0.95 for L_2 and 0.99 for IP. Cells show R/P/Q/D for L_2 and R/P/Q for IP, where R is recall, P precision, Q QPS, and D the mean number of distance calculations per query. Simp., H2-S, and SA-S abbreviate SIMPFER, H2-Simpfer, and SA-Simpfer. Boldface marks the best value in each row. Red brackets give ANQI's QPS speedup over the underlined fastest qualifying baseline. ‡ marks a missed target.

B. End-to-End Effectiveness

As the first and most important question, we ask whether ANQI is faster than baseline methods at the same recall. The five left columns of Fig. 4 show the complete frontiers, and Table IV lists each baseline's fastest measured point that meets the recall target. For ANQI, it lists the fastest target-reaching point whose precision is no more than 0.01 below that baseline.

a) L_2: Near recall 0.95, ANQI is 2.51–3.49× faster than HRNN across all three datasets and ranks. It uses 2.7k–15.5k complete distance evaluations per query, versus 6.6k–45.5k for HRNN. Other baselines require 89k–107M evaluations and often miss the target. ANQI instead verifies a high-recall pool obtained with less graph work. Even on 960-dimensional GIST1M, it sustains 2.53k–4.29k QPS with precision above 0.99. The margins on SIFT1M and DEEP1M are comparable, so the gain is not tied to one dimension.

b) Inner product: At recall 0.99, ANQI reaches precision 0.97–1.0 at 859–21.4k QPS, which is 17–2,046× faster than the fastest qualifying IP baseline. The specialized baselines produce isolated points at 0.50–50 QPS, and several exact-recall points fall to precision 0.56 by retaining too many candidates. ANQI, in contrast, sends a smaller pool to its compact verifier through threshold-aware routing, while rank-specific residuals preserve the boundary in the AKNN-derived matrix $\widetilde{T}[o,k] = -\widetilde{\mathrm{ip}}_k(o)$.

Overall, ANQI surpasses the baseline methods on quality-efficiency frontier over all evaluation settings. Across all 15 selected points, ANQI is 2.51–2050× faster than the fastest qualifying baseline, because it reduces candidate work before verification while meeting the metric-specific recall target and the precision tolerance used for selection. The advantage persists as the search budget increases: larger k requires retaining more true answers, yet the curves do not show a corresponding precision collapse.

c) Latency and batching: We next examine whether ANQI serves both latency-sensitive single requests and throughput-oriented batches. Table V separates isolated request latency from large-batch throughput. The measurements include query decode and report medians over three process repetitions. Across the five datasets and three ranks, one request on one thread takes 17–70 ms. With 64 workers, the largest measured batch amortizes to 0.017–1.6 ms per query. ANQI therefore retains practical isolated-request latency while supporting substantially higher throughput on batched workloads. Within a dataset, changing k shifts isolated latency by at most 18 ms, so multi-rank support adds limited rank-dependent control cost compared with the variation across dimensions and metrics. Because the two blocks change both concurrency and batch size, their ratio is not a pure batching speedup. The detailed protocol appears in Appendix A-I.

TABLE V
SINGLE-REQUEST LATENCY AND LARGE-BATCH AMORTIZED QUERY TIME (MS).

Dataset	One-thread single request k=10	50	100	64-thread largest batch / query k=10	50	100
SIFT1M	17	17	17	0.017	0.021	0.053
DEEP1M	17	17	17	0.017	0.021	0.023
GIST1M	25	25	30	0.35	0.44	0.75
Music100	18	21	28	0.076	0.22	0.23
MSMarco1M	70	52	65	1.5	1.1	1.6

The left block uses one query and one thread. The right uses 64 threads and the largest measured batch: 100 queries for Music100 and 1,000 otherwise.

C. Cost and Breakdown

Here we report the construction cost result on time and memroy and the breakdown analysis. Table VI reports build time and index size: ANQI builds 1.5–1.7× faster than the second best index HRNN on all million-scale settings, with smaller index size. Plain HNSW and the IP baselines build faster by shifting more offline construction work to high-recall online queries. ANQI instead pays once for a graph–verifier bundle

TABLE VI
BUILD TIME AND INDEX SIZE ON THE MILLION-SCALE DATASETS.

L_2				**IP**			
Dataset	Method	Build (s)	Index (GB)	Dataset	Method	Build (s)	Index (GB)
SIFT1M	ANQI	87.33	0.85	Music100	ANQI	686.59	0.71
	HRNN [4]	150.00	6.67		SAH [11]	17.73	0.50
	SFT/RDT [9], [10] HNSW	**14.00**	**0.66**		H2-Simpfer [1]	1.57	0.84
	HAMG [5]	69.00	20.53		SA-Simpfer [1]	**1.45**	**0.47**
DEEP1M	ANQI	102.59	0.73		SIMPFER [1]	8.31	0.64
	HRNN [4]	152.00	6.54	MSMarco1M	ANQI	607.77	3.47
	SFT/RDT [9], [10] HNSW	**12.00**	**0.53**		SAH [11]	125.97	0.62
	HAMG [5]	51.17	20.40		H2-Simpfer [1]	**5.61**	0.86
GIST1M	ANQI	392.83	4.25		SA-Simpfer [1]	5.94	**0.47**
	HRNN [4]	480.00	10.00		SIMPFER [1]	6.88	0.64
	SFT/RDT [9], [10] HNSW	**77.00**	**3.99**				
	HAMG [5]	308.00	23.86				

Index size is the persisted bytes of each method's index files. Baseline build times are measured wall-clock times, the programs' own timers, or times derived from log timestamps; where the selected configuration differs across k, the table shows the average over $k = 10, 50, 100$. Boldface marks each dataset's column minimum.

TABLE VII
DEEP1M BUILD TIME AND MATCHED-QUALITY QUERY COSTS.

Total build time: 102.6 s			**Query at matched quality**					
Component	Time	Share	k	Target	R/P	QPS	Graph D	Verify D
AKNN threshold generation	54.81	53.4%	10	.95	.96/1.00	88k	2.81k	100
			10	.99	.99/1.00	44k	5.16k	104
Rank-8 seed and LRQ fitting	33.75	32.9%	50	.95	.96/1.00	70k	3.31k	98
			50	.99	.99/1.00	37k	6.41k	100
Lift, graph, and persistence	14.03	13.7%	100	.95	.95/1.00	64k	3.65k	98
			100	.99	.99/1.00	29k	8.00k	100

shared by all three ranks. The measured tradeoff therefore fits static read-heavy collections better than frequently rebuilt ones.

We give a time breakdown analysis on ANQI's query work in Table VII, which reports mean graph and verification distance calculations. On DEEP1M, the table assigns 53% of build time to AKNN threshold generation, 33% to the verifier model, and 14% to lifting, graph construction, and persistence. At query time, graph work grows from 2.81k to 8.00k distance calculations as k or recall rises, while verification adds only 98–104 checks. High-recall cost therefore remains concentrated in the graph stage targeted by lifting.

D. Scalability from 10M to 1B Vectors

The success key of an index in industrial environment is its scalability on real-world large-scale workload. To test whether our design scales, the right column of Fig. 4 extends the benchmark by three orders of magnitude. Table VIII reports online work, build time, and index size.

At 10M, ANQI is $3.2\times$ faster than HRNN, builds $1.4\times$ faster, and has an index $73\times$ smaller. The HRNN diagnostic does not expose a comparable distance counter, so Table VIII leaves that field unavailable. We do not extend HRNN to 100M or 1B because its measured 10M index already occupies $86.7\,\mathrm{GB}$, whereas the corresponding ANQI index is $1.18\,\mathrm{GB}$.

Build time grows from 26 minutes at 10M to 5.8 hours at 100M and 53 hours at 1B, and the 100M and 1B totals include generating the precomputed AKNN streams. Index size grows

TABLE VIII
MEASURED SIFT SCALABILITY.

Scale	Method	R/P	Q/D	B/I
10M	ANQI	.95/1.00	35k/6.2k	26min/1.18 GB
	HRNN [4]	.96/1.00	11k/N/A	37min/86.7 GB
100M	ANQI	.95/1.00	14k/13k	5.8h/11.8 GB
1B	ANQI	.98/.97	17k/9.5k	53h/118 GB

Rows report the fastest measured points with recall and precision at least 0.95. ANQI points come from complete validation curves, and the 1B selection is independently audited on a fresh disjoint workload. Q/D denotes QPS and mean distance calculations. B/I denotes build time and index size.

from 1.18 to 11.8 and 118 GB, about 118 bytes per vector for the degree-20 graph, route cutoffs, and M8+u8 verifier state. Build time per vector stays within 0.16–0.21 ms, consistent with a bounded graph degree and a constant verifier state per object (eight coefficients and one 8-bit code for the single materialized rank). At 1B, exact reverse neighbors are infeasible to compute, so the validation and fresh workloads use a proxy ground truth derived from approximate forward top-10 lists. Against exact neighbors for 6,000 sampled base vectors, these lists have ID and tie-aware recall@10 of 0.99. Throughput remains above 10k QPS at every scale with recall and precision at least 0.95. Different construction and search budgets make work and QPS nonmonotone, so the result establishes full-pipeline feasibility.

E. Local Radius Law Examination in Euclidean Space

The Local Radius Law suggests that the cutoff columns of different ranks share a small common basis. To test this before any residual correction, an M-only diagnostic fits rank-8 PCA to 120,000 exact DEEP1M threshold rows over ranks 1–500. All objects receive eight coefficients from complete 500-rank rows. Because every tested $k = 10, \ldots, 300$ is represented, Panels A–B of Fig. 5 measure in-range compression rather than extrapolation. Full-scan evaluation of the reconstructed cutoffs over all base objects removes graph-search misses.

Mean relative error falls from 1.7% at $k = 10$ to 0.10% at $k = 300$, while its 99th percentile falls from 6.1% to 0.48%. Full-scan recall/precision rises from $0.95/0.96$ to $1.00/1.00$. Thus eight coefficients capture most cross-rank structure in 32 bytes per vector. The falling error is consistent with larger neighborhoods stabilizing local-density estimates. Underestimates account for 49–51% of reconstructions, so the shared term does not systematically expand or shrink the boundary. LRQ is most useful at the small-rank tail, where the remaining error is largest.

F. Why ANQI Works: Ablation Study on Lifting and Neighborhood Quantization

In this section we present ablation studies on ANQI for each of out index design. The matched controls in Fig. 5 follow the query pipeline and isolate candidate generation, candidate verification, and multi-rank reuse.

a) Candidate generation: Lift targets the routing cost of Section I. For $k = 10/100$, it reduces graph-distance work by $1.4/2.5\times$ at recall 0.95 and $2.8/3.5\times$ at recall 0.99, with precision 1.0 (Panels G–H). The advantage widens near

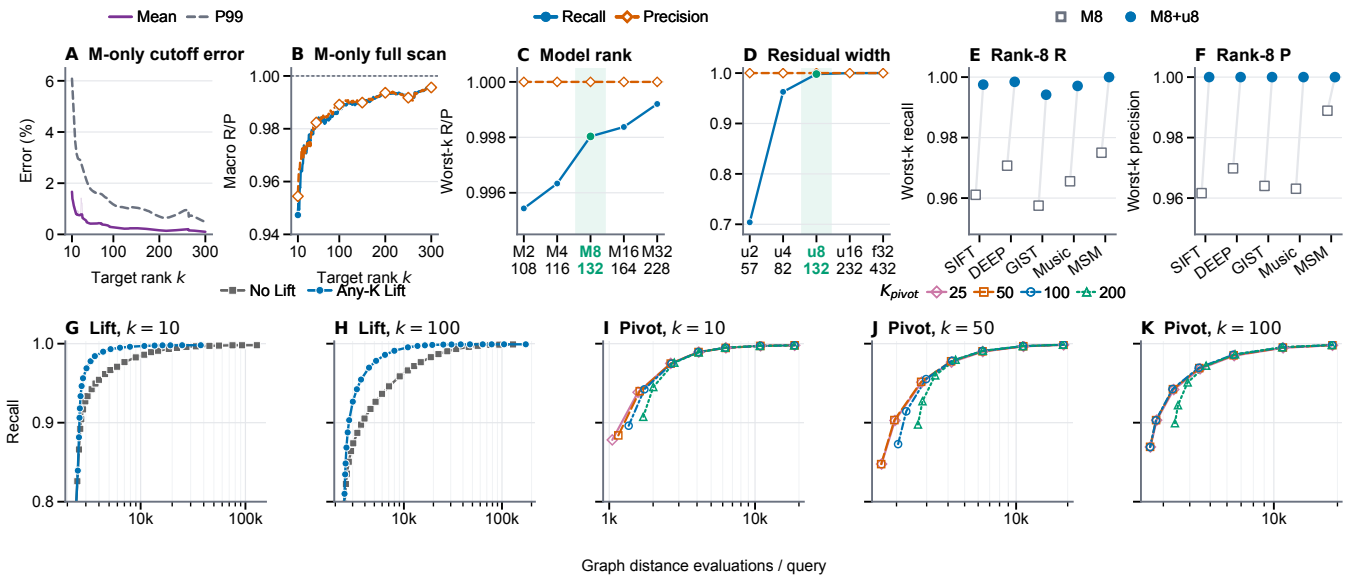


Fig. 5. Why ANQI works. (A–B) The M-only Local Radius Law diagnostic measures cutoff reconstruction and full-scan verifier quality. (C–D) Model-rank and residual-width sweeps identify M8+u8 as the compact-verifier knee. (E–F) The same rank-8 representation is evaluated across all five million-scale datasets. (G–H) Reverse-aware lifting reduces candidate-generation work. (I–K) One graph remains robust across construction pivots 25–200 and target ranks 10, 50, and 100. Except for the offline exact-threshold diagnostic in A–B, all panels use the deployed approximate-AKNN threshold source.

recall 0.99, when base-space routing must expand farther across different reverse boundaries, whereas encoding those boundaries in the navigation score reaches the same recall with less graph work. Full-precision cutoffs produce identical work and recall within 0.002, which places the gain in navigation rather than quantization. Compared with the no-lift control, Lift adds 1.2 seconds of build time and about 4 MB of index size.

b) Candidate verification: The verifier targets the cutoff cost of Section I. With candidate work and u8 fixed, raising M from 2 to 8 lifts worst-k recall from 0.996 to 0.998, while M16/M32 add less than 0.002 (Panels C–D). At M8, u2/u4 reach 0.71/0.96 recall, u8 reaches $0.998/1.0$ recall/precision at 132 B/vector, and u16/float32 approach 1.0 recall at 232/432 B/vector. M8+u8 is therefore the storage–quality knee, preserving boundary fidelity at one third of the 400 B/vector dense matrix.

To check that M8+u8 is not specific to one dataset, we apply it to all five million-scale datasets and $k \in \{10, 50, 100\}$. M8 without residuals falls to worst-case recall/precision $0.957/0.962$, and adding u8 raises the lowest recall to 0.994 and precision to 1.0 on every dataset (Panels E–F). The shared rank-8 component therefore captures the common cross-rank structure, while the residual codes retain dataset- and rank-specific boundary detail, and the same capacity works on GIST1M and both IP datasets.

c) Multi-rank reuse: Every pivot 25–200 reaches recall 0.99 for all three ranks with target-k collection, including $K_{\text{pivot}} = 25$ at $k = 100$ (Panels I–K). At $k = 100$, all pivots first reach recall 0.99 together at about 11k graph-distance evaluations on this sweep's coarse search grid, while the finer grid of Table VII reaches it at 8.00k for $K_{\text{pivot}} = 100$. Candidate counts also remain near 11, 50, and 100, and build time and index size without the verifier stay at 66–69 seconds and about 0.60 GB for every pivot. This overlap shows that target-rank collection and score correction govern behavior across the tested k and K_{pivot}, not their equality. Since the choice matters little, we use $K_{\text{pivot}} = 100$ to match the largest materialized rank.

These controls locate distinct loss mechanisms. Lifting changes candidate coverage, M8+u8 changes boundary decisions, and the pivot sweep tests cross-rank exposure.

VI. Conclusion

In this paper, we present ANQI, an index for reverse k-NN search that turns AKNN-derived reverse cutoffs into reusable index state. We design a lifted graph to reduce candidate-generation work and we propose a compact LRQ mechanism to replace full-precision rank-specific tables, allowing ANQI to scale to 1B vectors. Together, they separate candidate coverage from boundary fidelity while sharing one index across the represented ranks.

Across five million-scale datasets, ANQI delivers 2.51–2050× high-recall speedups with comparable precision. On SIFT1B, it reaches .98/.97 recall/precision and 17 k QPS on a serve node. Ablations show complementary roles: lifting reduces navigation work, LRQ preserves boundary detail, and cross-rank reuse avoids rebuilding the graph. ANQI currently indexes a fixed vector collection for Euclidean distance and inner product, serves the ranks materialized at build time, and spends the largest share of build time on the AKNN threshold pass. Future work therefore includes incremental updates, other similarity functions, rank reuse beyond the construction rank for inner product, and a cheaper threshold pass.

REFERENCES

[1] D. Amagata and T. Hara, "Reverse maximum inner product search: How to efficiently find users who would like to buy my item?" in *Proceedings of the 15th ACM Conference on Recommender Systems*, 2021, pp. 273–281.

[2] ——, "Reverse maximum inner product search: Formulation, algorithms, and analysis," *ACM Transactions on the Web*, vol. 17, no. 4, pp. 1–23, 2023.

[3] F. Korn and S. Muthukrishnan, "Influence sets based on reverse nearest neighbor queries," *ACM SIGMOD Record*, vol. 29, no. 2, pp. 201–212, 2000.

[4] W. Xia, M. Yang, W. Li, and W. Wang, "HRNN: A hybrid graph index for approximate reverse k-nearest neighbor search on high-dimensional vectors," *arXiv preprint arXiv:2606.03225*, 2026.

[5] Y. Song, K. Wang, B. Yao, Z. Chen, J. Xie, and F. Li, "Efficient reverse k approximate nearest neighbor search over high-dimensional vectors," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, 2024, pp. 4262–4274.

[6] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[7] C. Fu, C. Xiang, C. Wang, and D. Cai, "Fast approximate nearest neighbor search with the navigating spreading-out graph," *Proceedings of the VLDB Endowment*, vol. 12, no. 5, pp. 461–474, 2019.

[8] S. Jayaram Subramanya, Devvrit, H. V. Simhadri, R. Krishnaswamy, and R. Kadekodi, "DiskANN: Fast accurate billion-point nearest neighbor search on a single node," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[9] A. Singh, H. Ferhatosmanoglu, and A. Ş. Tosun, "High dimensional reverse nearest neighbor queries," in *Proceedings of the 12th International Conference on Information and Knowledge Management*, 2003, pp. 91–98.

[10] G. Casanova, E. Englmeier, M. E. Houle, P. Kröger, M. Nett, E. Schubert, and A. Zimek, "Dimensional testing for reverse k-nearest neighbor search," *Proceedings of the VLDB Endowment*, vol. 10, no. 7, pp. 769–780, 2017.

[11] Q. Huang, Y. Wang, and A. K. H. Tung, "SAH: Shifting-aware asymmetric hashing for reverse k maximum inner product search," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 4, 2023, pp. 4312–4321.

[12] P. Indyk and R. Motwani, "Approximate nearest neighbors: towards removing the curse of dimensionality," in *Proceedings of the 30th Annual ACM Symposium on Theory of Computing*, 1998, pp. 604–613.

[13] M. Datar, N. Immorlica, P. Indyk, and V. S. Mirrokni, "Locality-sensitive hashing scheme based on p-stable distributions," in *Proceedings of the 20th Annual Symposium on Computational Geometry*, 2004, pp. 253–262.

[14] A. Andoni and P. Indyk, "Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions," *Communications of the ACM*, vol. 51, no. 1, pp. 117–122, 2008.

[15] M. Muja and D. G. Lowe, "Fast approximate nearest neighbors with automatic algorithm configuration," in *Proceedings of the International Conference on Computer Vision Theory and Applications*, vol. 1, 2009, pp. 331–340.

[16] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[17] K. Hajebi, Y. Abbasi-Yadkori, H. Shahbazi, and H. Zhang, "Fast approximate nearest-neighbor search with k-nearest neighbor graph," in *Proceedings of the 22nd International Joint Conference on Artificial Intelligence*, 2011, pp. 1312–1316.

[18] W. Dong, M. Charikar, and K. Li, "Efficient k-nearest neighbor graph construction for generic similarity measures," in *Proceedings of the 20th International Conference on World Wide Web*, 2011, pp. 577–586.

[19] C. Fu and D. Cai, "EFANNA: An extremely fast approximate nearest neighbor search algorithm based on kNN graph," *arXiv preprint arXiv:1609.07228*, 2016.

[20] B. Harwood and T. Drummond, "FANNG: Fast approximate nearest neighbour graphs," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 5713–5722.

[21] M. Iwasaki, "Pruned bi-directed k-nearest neighbor graph for proximity search," in *Proceedings of the International Conference on Similarity Search and Applications*, 2016, pp. 20–33.

[22] M. Iwasaki and D. Miyazaki, "Optimization of indexing based on k-nearest neighbor graph for proximity search in high-dimensional data," *arXiv preprint arXiv:1810.07355*, 2018.

[23] Y. Fu, C. Chen, Y. Chen, W.-F. Wong, and B. He, "Vista: Vector indexing and search for large-scale imbalanced datasets," in *2025 IEEE 41st International Conference on Data Engineering (ICDE)*, 2025, pp. 543–556.

[24] M. D. Manohar, Z. Shen, G. Blelloch, L. Dhulipala, Y. Gu, H. V. Simhadri, and Y. Sun, "ParlayANN: Scalable and deterministic parallel graph-based approximate nearest neighbor search algorithms," in *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, 2024, pp. 270–285.

[25] M. D. Manohar, T. Kim, and G. E. Blelloch, "Range retrieval with graph-based indices," *arXiv preprint arXiv:2502.13245*, 2025.

[26] Y. Tao, D. Papadias, X. Lian, and X. Xiao, "Multidimensional reverse kNN search," *The VLDB Journal*, vol. 16, no. 3, pp. 293–316, 2007.

[27] E. Achtert, C. Böhm, P. Kröger, P. Kunath, A. Pryakhin, and M. Renz, "Efficient reverse k-nearest neighbor search in arbitrary metric spaces," in *Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data*, 2006, pp. 515–526.

[28] Z. Bai, P. Chen, and M. Wahib, "RT-RkNN: Reverse k nearest neighbor queries as a graphics ray casting problem," *arXiv preprint arXiv:2605.26671*, 2026.

[29] F. Aurenhammer, "Power diagrams: properties, algorithms and applications," *SIAM Journal on Computing*, vol. 16, no. 1, pp. 78–96, 1987.

[30] O. Cheong, A. Vigneron, and J. Yon, "Reverse nearest neighbor queries in fixed dimension," *International Journal of Computational Geometry & Applications*, vol. 21, no. 2, pp. 179–188, 2011.

[31] D. Arthur and S. Y. Oudot, "Reverse nearest neighbors search in high dimensions using locality-sensitive hashing," *arXiv preprint arXiv:1011.4955*, 2010.

[32] A. Shrivastava and P. Li, "Asymmetric LSH (ALSH) for sublinear time maximum inner product search (MIPS)," *Advances in Neural Information Processing Systems*, vol. 27, 2014.

[33] B. Neyshabur and N. Srebro, "On symmetric and asymmetric LSHs for inner product search," in *Proceedings of the 32nd International Conference on Machine Learning*, 2015, pp. 1926–1934.

[34] A. Shrivastava and P. Li, "Improved asymmetric locality sensitive hashing (ALSH) for maximum inner product search (MIPS)," *arXiv preprint arXiv:1410.5410*, 2014.

[35] X. Yan, J. Li, X. Dai, H. Chen, and J. Cheng, "Norm-ranging LSH for maximum inner product search," *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[36] X. Zhao, B. Zheng, X. Yi, X. Luan, C. Xie, X. Zhou, and C. S. Jensen, "FARGO: Fast maximum inner product search via global multi-probing," *Proceedings of the VLDB Endowment*, vol. 16, no. 5, pp. 1100–1112, 2023.

[37] S. Morozov and A. Babenko, "Non-metric similarity graphs for maximum inner product search," *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[38] J. Liu, X. Yan, X. Dai, Z. Li, J. Cheng, and M.-C. Yang, "Understanding and improving proximity graph based maximum inner product search," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 1, 2020, pp. 139–146.

[39] S. Tan, Z. Xu, W. Zhao, H. Fei, Z. Zhou, and P. Li, "Norm adjusted proximity graph for fast inner product retrieval," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 2021, pp. 1552–1560.

[40] Z. Zhou, S. Tan, Z. Xu, and P. Li, "Möbius transformation for fast inner product search on graph," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[41] T. Chen, C. Fu, K. Wang, X. Ke, Y. Gao, W. Zhou, Y. Ni, and A. Zeng, "Maximum inner product is query-scaled nearest neighbor," *Proceedings of the VLDB Endowment*, vol. 18, no. 6, pp. 1770–1783, 2025.

[42] T. Chen, C. Fu, X. Ke, Y. Gao, Y. Ni, and A. Zeng, "Stitching inner product and Euclidean metrics for topology-aware maximum inner product search," in *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2025, pp. 2341–2350.

[43] E. Achtert, C. Böhm, P. Kröger, P. Kunath, A. Pryakhin, and M. Renz, "Approximate reverse k-nearest neighbor queries in general metric spaces," in *Proceedings of the 15th ACM International Conference on Information and Knowledge Management*, 2006, pp. 788–789.

[44] M. Berrendorf, F. Borutta, and P. Kröger, "k-distance approximation for memory-efficient RkNN retrieval," in *Proceedings of the International Conference on Similarity Search and Applications*, 2019, pp. 57–71.

[45] S. Obermeier, M. Berrendorf, and P. Kröger, "Memory-efficient RkNN retrieval by nonlinear k-distance approximation," in *2020 IEEE/WIC/ACM International Joint Conference on Web Intelligence and Intelligent Agent Technology (WI-IAT)*, 2020, pp. 387–390.

[46] C. Eckart and G. Young, "The approximation of one matrix by another of lower rank," *Psychometrika*, vol. 1, no. 3, pp. 211–218, 1936.

[47] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011.

[48] Y. Chen, T. Guan, and C. Wang, "Approximate nearest neighbor search by residual vector quantization," *Sensors*, vol. 10, no. 12, pp. 11 259–11 273, 2010.

[49] T. Ge, K. He, Q. Ke, and J. Sun, "Optimized product quantization for approximate nearest neighbor search," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2013, pp. 2946–2953.

[50] Y. Kalantidis and Y. Avrithis, "Locally optimized product quantization for approximate nearest neighbor search," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2014, pp. 2321–2328.

[51] A. Babenko and V. Lempitsky, "Additive quantization for extreme vector compression," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2014, pp. 931–938.

[52] T. Zhang, C. Du, and J. Wang, "Composite quantization for approximate nearest neighbor search," in *Proceedings of the 31st International Conference on Machine Learning*, 2014, pp. 838–846.

[53] R. Guo, P. Sun, E. Lindgren, Q. Geng, D. Simcha, F. Chern, and S. Kumar, "Accelerating large-scale inference with anisotropic vector quantization," in *Proceedings of the 37th International Conference on Machine Learning*, 2020, pp. 3887–3896.

[54] Y. Bachrach, Y. Finkelstein, R. Gilad-Bachrach, L. Katzir, N. Koenigstein, N. Nice, and U. Paquet, "Speeding up the Xbox recommender system using a Euclidean transformation for inner-product spaces," in *Proceedings of the 8th ACM Conference on Recommender Systems*, 2014, pp. 257–264.

[55] D. O. Loftsgaarden and C. P. Quesenberry, "A nonparametric estimate of a multivariate density function," *The Annals of Mathematical Statistics*, vol. 36, no. 3, pp. 1049–1051, 1965.

[56] K. Fukunaga and L. Hostetler, "Optimization of k nearest neighbor density estimates," *IEEE Transactions on Information Theory*, vol. 19, no. 3, pp. 320–326, 1973.

[57] P. J. Bickel and L. Breiman, "Sums of functions of nearest neighbor distances, moment bounds, limit theorems and a goodness of fit test," *The Annals of Probability*, vol. 11, no. 1, pp. 185–214, 1983.

[58] E. Levina and P. Bickel, "Maximum likelihood estimation of intrinsic dimension," *Advances in Neural Information Processing Systems*, vol. 17, 2004.

[59] E. Facco, M. d'Errico, A. Rodriguez, and A. Laio, "Estimating the intrinsic dimension of datasets by a minimal neighborhood information," *Scientific Reports*, vol. 7, no. 1, p. 12140, 2017.

APPENDIX A
PROOFS AND SUPPLEMENTARY ANALYSIS

This appendix supplies the full arguments behind the mathematical statements in Sections III and IV. The lifting, score-correction, set-containment, and recall-factorization results are exact under their stated premises. The Local Radius Law and the bounds derived from it, in contrast, are conditional population analyses: they explain threshold structure and verifier error, but do not guarantee coverage by a finite ANN traversal. We use exact thresholds and real arithmetic unless a statement says otherwise.

A. Exact lifting, shared envelopes, and score correction

This subsection separates three facts. Lifting preserves reverse membership and score order. Rank monotonicity makes a construction-rank acceptance set an exact envelope for smaller target ranks. Target-rank score correction remains an algebraic identity for any represented rank. None of these statements alone guarantees that a finite graph search reaches the whole relevant envelope.

Proof of Theorem 1. We first verify the predicate in the original score space. For L_2, (6) gives

$$\begin{aligned}\|q-o\|^2 \le r_k(o)^2 &\iff \|q\|^2 + \|o\|^2 - 2\cdot\langle q,o\rangle \le r_k(o)^2\\ &\iff 2\cdot\langle q,o\rangle \ge \|q\|^2 + \|o\|^2 - r_k(o)^2\\ &\iff \langle q,o\rangle - \tfrac{1}{2}\cdot(\|o\|^2 - r_k(o)^2) \ge \tfrac{1}{2}\cdot\|q\|^2.\end{aligned}$$

The left-hand side is exactly $s_k(q,o) = \langle q^\ell, o_k^\ell\rangle$ for the L_2 row in (8), and the right-hand side is $\tau(q)$. For inner product, the same lifted inner product is $s_k(q,o) = \langle q,o\rangle - \mathrm{ip}_k(o)$, so

$$o \in A_k(q) \iff s_k(q,o) \ge 0 = \tau(q).$$

Thus, for either metric,

$$o \in A_k(q) \iff s_k(q,o) \ge \tau(q).$$

Now consider the norm-equalized vectors in (10). The last coordinate is real because $M_\star$ is at least the largest norm of the lifted indexed vectors. Hence $\|\bar o_k\|^2 = \|o_k^\ell\|^2 + M_\star^2 - \|o_k^\ell\|^2 = M_\star^2$. Using $\bar q = [q^\ell, 0]$ and expanding the squared distance gives

$$\begin{aligned}\|\bar q - \bar o_k\|^2 &= \|\bar q\|^2 + \|\bar o_k\|^2 - 2\cdot\langle \bar q, \bar o_k\rangle\\ &= \|q^\ell\|^2 + M_\star^2 - 2\cdot\langle q^\ell, o_k^\ell\rangle\\ &= \|q^\ell\|^2 + M_\star^2 - 2\cdot s_k(q,o).\end{aligned}$$

Since the first two terms are independent of o, a larger score is equivalent to a smaller Euclidean distance. Substituting $s_k(q,o) \ge \tau(q)$ into this identity yields

$$\begin{aligned}s_k(q,o) \ge \tau(q) &\iff \|\bar q - \bar o_k\|^2 \le \|q^\ell\|^2 + M_\star^2 - 2\cdot\tau(q)\\ &\iff \|\bar q - \bar o_k\|^2 \le \rho_k(q)^2,\end{aligned}$$

which proves the lifted-ball equivalence and the score-ordering claim. Finally, for any candidate set $C(q)$, applying this pointwise equivalence only to vectors in $C(q)$ gives

$$\{o \in C(q) : \|\bar q - \bar o_k\|^2 \le \rho_k(q)^2\} = C(q) \cap A_k(q).$$

This is a statement about filtering the supplied candidates. It does not claim that a finite graph traversal necessarily supplies every member of $A_k(q)$. □

Proof of Proposition 4. For the L_2 case, define $N_o(r) = |\{x \in \mathcal{X}\setminus\{o\} : \|x-o\| \le r\}|$. By definition, $r_k(o)$ is a kth-neighbor radius, so $N_o(r_k(o)) \ge k$. If $K \ge k$, the Kth order statistic cannot be smaller than the kth one:

$$r_k(o) \le r_K(o).$$

Therefore, whenever $\|q-o\| \le r_k(o)$, we also have $\|q-o\| \le r_K(o)$, proving $A_k(q) \subseteq A_K(q)$.

For inner product, let the scores $\{\langle x,o\rangle : x \ne o\}$ be sorted in nonincreasing order. The kth order statistic satisfies $\mathrm{ip}_k(o) \ge \mathrm{ip}_K(o)$ for $K \ge k$. Hence

$$\langle q,o\rangle \ge \mathrm{ip}_k(o) \quad\Longrightarrow\quad \langle q,o\rangle \ge \mathrm{ip}_K(o),$$

which proves the same set containment. Theorem 1 maps each of these pointwise predicates to its lifted-ball predicate, so the construction-rank acceptance region at K is a valid exact candidate envelope for every represented target rank $k \le K$. The proposition does not by itself provide a finite-beam graph-coverage guarantee. □

The shared-rank result concerns acceptance sets, not graph reachability: the construction-rank set is an exact candidate envelope for each smaller represented rank. The experiments measure how well finite graph traversal covers that envelope.

Proof of Lemma 1. For an arbitrary object-side cutoff $a(v)$, the score is

$$s_a(q,v) = \langle q,v\rangle - \tfrac{1}{2}\cdot\big(\|v\|^2 - a(v)\big).$$

Adding the correction to another cutoff $b(v)$ gives

$$\begin{aligned}&s_a(q,v) + \tfrac{1}{2}\cdot\big(b(v) - a(v)\big)\\ &= \langle q,v\rangle - \tfrac{1}{2}\cdot\|v\|^2 + \tfrac{1}{2}\cdot a(v) + \tfrac{1}{2}\cdot b(v) - \tfrac{1}{2}\cdot a(v)\\ &= \langle q,v\rangle - \tfrac{1}{2}\cdot\big(\|v\|^2 - b(v)\big) = s_b(q,v).\end{aligned}$$

The identity is algebraic and holds for any represented rank, including a rank larger than K_{pivot}. The exact analysis sets $a = t_{K_{\mathrm{pivot}}}$ and $b = t_k$. The deployed path instead sets $a = \widetilde{t}_{K_{\mathrm{pivot}}}$ and $b = \hat t_k$, so it uses the same substitution without reading either true threshold. What remains empirical beyond the construction pivot is whether this approximate envelope and the finite graph traversal cover the target answer set. □

B. Euclidean Local Radius Law and shell growth

The Local Radius Law is an idealized population statement. It identifies the rank-k radius by $N_o(r_k(o)) = k$ within a neighborhood where density and intrinsic dimension are stable. Under this premise, squared radius is a power of rank. A Taylor expansion then separates rank functions from object-specific coefficients, which motivates a shared threshold basis. It does not assert that a finite sample follows one power law at every rank.

Proof of Proposition 1. Under the local ball-mass model,

$$N_o(r) = \rho(o)\cdot V_{d(o)}\cdot r^{d(o)}.$$

The model identifies the kth-neighbor radius by $N_o(r_k(o)) = k$, so

$$r_k(o)^{d(o)} = \frac{k}{\rho(o)\cdot V_{d(o)}}.$$

Raising both sides to the power $2/d(o)$ gives

$$\begin{aligned} r_k(o)^2 &= \left(\frac{k}{\rho(o)\cdot V_{d(o)}}\right)^{2/d(o)} \\ &= \left[\rho(o)\cdot V_{d(o)}\right]^{-2/d(o)} \cdot k^{2/d(o)} \\ &= \gamma(o)\cdot k^{\nu(o)}. \end{aligned}$$

with $\nu(o) = 2/d(o)$ and $\gamma(o) = [\rho(o)\cdot V_{d(o)}]^{-\nu(o)}$.

Write $\nu(o) = \nu_0 + \beta(o)$. Then

$$k^{\nu(o)} = k^{\nu_0+\beta(o)} = k^{\nu_0}\cdot\exp\bigl(\beta(o)\cdot\ln k\bigr).$$

For $x = \beta(o)\cdot\ln k$, the exponential power series is

$$e^x = \sum_{m=0}^{\infty}\frac{x^m}{m!}.$$

Multiplying by $\gamma(o)\cdot k^{\nu_0}$ and inserting $x = \beta(o)\cdot\ln k$ produces

$$r_k(o)^2 = \sum_{m=0}^{\infty}\underbrace{\frac{\gamma(o)\cdot\beta(o)^m}{m!}}_{f_m(o)}\underbrace{k^{\nu_0}\cdot(\ln k)^m}_{\phi_m(k)}.$$

Truncating after degree p gives

$$r_k(o)^2 = \sum_{m=0}^{p} f_m(o)\cdot\phi_m(k) + R_p(o,k),$$

and Taylor's theorem bounds the remainder by

$$|R_p(o,k)| \le \gamma(o)\cdot k^{\nu_0}\cdot e^{|\beta(o)\cdot\ln k|}\cdot\frac{|\beta(o)\cdot\ln k|^{p+1}}{(p+1)!}.$$

The factorization supports a small shared rank basis when $|\beta(o)\cdot\ln k|$ is moderate. This is an object-local approximation: changes in density, curvature, or intrinsic dimension can invalidate one exponent over a wider rank interval, and no claim about the ambient dimension D is needed for the derivation. □

Proof of Corollary 1. Applying (13) at k and K gives

$$r_k(o)^2 = \gamma(o)\cdot k^{\nu(o)}, \qquad r_K(o)^2 = \gamma(o)\cdot K^{\nu(o)}.$$

Because $\gamma(o) > 0$, division cancels the object-specific intensity and volume factor:

$$\frac{r_k(o)^2}{r_K(o)^2} = \left(\frac{k}{K}\right)^{\nu(o)}.$$

Within the same smooth population model, ties have zero probability and the nominal rank masses satisfy $N_o(r_k(o)) = k$ and $N_o(r_K(o)) = K$. The shell between the two radii therefore has mass

$$N_o(r_k(o)) - N_o(r_K(o)) = k - K.$$

This is a population-model identity, not a finite-sample cardinality claim. Ties, density changes, or a changing intrinsic dimension can alter the observed shell. □

C. Inner-product projection quantiles

Inner-product thresholds are directional projection quantiles, not Euclidean radii. The identity below characterizes the exact signed threshold matrix defined by the reverse predicate. It neither invokes the Local Radius Law nor proves that the matrix is low rank. The deployed low-rank-plus-LRQ representation is fitted empirically to the corresponding AKNN-derived signed thresholds.

Proof of Proposition 2. Suppose first that $o \neq 0$ and define $u_o = o/\|o\|$. For every base vector x, bilinearity of the inner product gives

$$\langle x, o\rangle = \langle x, \|o\|u_o\rangle = \|o\|\cdot\langle x, u_o\rangle.$$

The multiplier $\|o\|$ is positive and is the same for every x, so it preserves the ordering of all projection scores. Consequently, the kth largest inner product equals the same positive multiplier times the kth largest projection:

$$\mathrm{ip}_k(o) = \|o\|\cdot\pi_k(o).$$

The reverse predicate becomes

$$\begin{aligned} \langle q, o\rangle \ge \mathrm{ip}_k(o) &\iff \|o\|\cdot\langle q, u_o\rangle \ge \|o\|\cdot\pi_k(o) \\ &\iff \langle q, u_o\rangle \ge \pi_k(o), \end{aligned}$$

where the second equivalence divides by the positive number $\|o\|$. Since $T[o,k] = -\mathrm{ip}_k(o)$ by definition, the signed threshold row is $T[o,k] = -\|o\|\cdot\pi_k(o)$. If $o = 0$, every inner product with o is zero. all order statistics and the signed threshold row are therefore zero, which is the stated special case. □

D. One-sided verification

This result isolates threshold reconstruction from candidate generation. It applies to a full scan or to any fixed candidate set. Restrictiveness or expansiveness is defined relative to the true threshold t_k. Floor decoding is restrictive relative to its AKNN source $\widetilde{t}_k$, but that fact alone does not order $\hat{t}_k$ against t_k. If a reconstructed threshold also controls candidate collection, as in the cross-pivot path, its error can additionally change which candidates the graph reaches.

Proof of Proposition 3. Assume first that the reconstruction is restrictive, so $\hat{t}_k(o) \le t_k(o)$ for every o. If an object is accepted by the represented verifier, then

$$\kappa(q,o) \le \hat{t}_k(o) \le t_k(o),$$

and it is therefore also in the exact answer. This proves $\hat{A}_k(q) \subseteq A_k(q)$ and rules out false positives. If an exact member is rejected, it satisfies the exact acceptance inequality but violates the represented one, namely

$$\hat{t}_k(o) < \kappa(q,o) \le t_k(o).$$

Thus every false negative lies in the displayed boundary band, and objects outside that band have the same decision under both thresholds.

For the expansive case, $\hat{t}_k(o) \ge t_k(o)$. Every exact member then obeys

$$\kappa(q,o) \le t_k(o) \le \hat{t}_k(o),$$

so $A_k(q) \subseteq \hat{A}_k(q)$ and verification cannot introduce false negatives. The only possible errors are objects in the opposite boundary band $\{o : t_k(o) < \kappa(q,o) \le \hat{t}_k(o)\}$, which can be false positives. □

E. Verifier recall under the local population model

The next calculation translates relative squared-radius error into an expected pooled-recall bound. It requires one assumption beyond the Local Radius Law: queries are drawn from the same local population used to model base-vector mass.

The result is not a deterministic bound on per-query MacroR, which is the metric reported in the experiments.

For each object o, suppose the query distribution satisfies

$$\Pr[q \in B(o,r)] = a(o) \cdot r^{d(o)}$$

over the radii considered, where $a(o) > 0$ absorbs local density and unit-ball volume. Assume a restrictive L_2 reconstruction with $0 \le \hat{r}_k(o)^2 \le r_k(o)^2$, and let

$$\epsilon_k = \max_o \frac{r_k(o)^2 - \hat{r}_k(o)^2}{r_k(o)^2} < 1.$$

Then, for every object,

$$\hat{r}_k(o)^2 \ge (1-\epsilon_k) \cdot r_k(o)^2, \qquad \hat{r}_k(o) \ge \sqrt{1-\epsilon_k}\, r_k(o).$$

For each object, the represented ball retains at least the following fraction of the exact local probability mass:

$$\begin{aligned} \frac{\Pr[q \in B(o, \hat{r}_k(o))]}{\Pr[q \in B(o, r_k(o))]} &= \left(\frac{\hat{r}_k(o)}{r_k(o)}\right)^{d(o)} \\ &\ge (1-\epsilon_k)^{d(o)/2} \\ &\ge (1-\epsilon_k)^{d_{\max}/2}, \end{aligned}$$

where $d_{\max} = \max_o d(o)$ and $0 < 1-\epsilon_k < 1$. Summing the numerator and denominator over objects gives

$$\begin{aligned} \mathrm{Recall}_{\mathrm{pool}} &:= \frac{\mathbb{E}_q[|\hat{A}_k(q)|]}{\mathbb{E}_q[|A_k(q)|]} \\ &\ge (1-\epsilon_k)^{d_{\max}/2}. \end{aligned} \tag{25}$$

This is a full-collection verifier bound under the stated population model. It does not include exact answers that a finite graph traversal never supplies.

F. Quantization width and threshold error

This section separates the added LRQ quantization error from the AKNN source error. A smaller codebook spacing moves $\hat{t}_k$ closer to $\widetilde{t}_k$. Its effect on exact-answer recall also depends on how $\widetilde{t}_k$ differs from t_k.

The one-sided property of floor decoding can be related directly to residual-codebook spacing. Let $J_k = |\mathcal{B}_k| \ge 2$, let the ordered entries be $b_{0,k} < \cdots < b_{J_k-1,k}$, and assume that every encoded residual lies in $[b_{0,k}, b_{J_k-1,k}]$. Define the maximum adjacent spacing by

$$\Delta_k = \max_{0 \le j < J_k - 1} (b_{j+1,k} - b_{j,k}).$$

For a residual in the interval $[b_{j,k}, b_{j+1,k})$, floor decoding selects $b_{j,k}$, so the following bound holds:

$$0 \le e_{o,k} - \tilde{e}_{o,k} < \Delta_k, \qquad 0 \le \widetilde{t}_k(o) - \hat{t}_k(o) < \Delta_k. \tag{26}$$

The second inequality follows by subtracting the shared mean and basis terms in (15) and (16). Residuals equal to the largest codebook endpoint have zero error. Including that endpoint handles the boundary case omitted by the open interval above.

For positive Euclidean thresholds, define $r^2_{k,\min} = \min_o t_k(o) > 0$. Suppose the AKNN source has relative error $|t_k(o) - \widetilde{t}_k(o)|/t_k(o) \le \eta_k$ for every object and the final reconstruction is restrictive relative to t_k. The triangle inequality and (26) then give

$$\epsilon_k = \max_o \frac{t_k(o) - \hat{t}_k(o)}{t_k(o)} \le \eta_k + \frac{\Delta_k}{r^2_{k,\min}}.$$

If the right-hand side is below one, the query-population model from Appendix A-E gives the conservative pooled bound

$$\mathrm{Recall}_{\mathrm{pool}} \ge \left(1 - \eta_k - \frac{\Delta_k}{r^2_{k,\min}}\right)^{d_{\max}/2}. \tag{27}$$

Thus Δ_k controls only the additional compression term, while η_k accounts for AKNN threshold estimation. This is an expected population bound under the stated ordering assumptions, not a finite-sample or MacroR guarantee. The verifier curves provide the empirical check.

G. Candidate coverage and verifier retention

For a query with nonempty exact answer $A = A_k(q)$, let $C = C_k(q)$ be the graph candidate set and let $\widehat{R} = \widehat{R}_k(q) \subseteq C$ be the verified output. The two loss mechanisms admit the exact decomposition

$$\begin{aligned} \mathrm{Recall}(q) &= \frac{|A \cap \widehat{R}|}{|A|} \\ &= \underbrace{\frac{|A \cap C|}{|A|}}_{\text{candidate coverage}} \quad \underbrace{\frac{|A \cap \widehat{R}|}{|A \cap C|}}_{\text{verifier retention}}, \end{aligned} \tag{28}$$

where the second factor is defined as zero when $|A \cap C| = 0$. For a fixed candidate set, exact thresholds make verifier retention one. A restrictive floor-decoded threshold can reduce only the second factor, and only through the boundary band in Proposition 3. In the cross-pivot path, however, the decoded threshold also controls the collection envelope. Its error can then change $C_k(q)$ and therefore the first factor as well.

This distinction explains the experimental controls. The lift and graph ablations measure candidate generation, while the M-only and verifier-capacity controls isolate threshold representation with a fixed full-scan candidate source.

H. Conditional completeness beyond the construction pivot

Exact target-rank score correction does not imply graph coverage. The next proposition states a sufficient reachability condition, which ANQI does not prove for a finite construction-rank graph.

For the L_2 path with exact thresholds, define

$$\begin{aligned} E_k(q) = \{v : s_{K_{\mathrm{pivot}}}(q,v) + \tfrac{1}{2} \cdot \big(t_k(v) - t_{K_{\mathrm{pivot}}}(v)\big) \\ \ge \tfrac{1}{2} \cdot \|q\|^2\}. \end{aligned}$$

Let $S_k(q) \subseteq E_k(q)$ be the envelope vertices encountered during the bounded descent stage of Algorithm 2.

Proposition 5 (Conditional cross-pivot completeness)**.** *Assume exact thresholds and arithmetic. If every $v \in E_k(q)$ is reachable from $S_k(q)$ along a directed path contained in $E_k(q)$, and the FIFO expands every enqueued envelope vertex, then ANQI returns the exact target-k answer for any represented k, including $k > K_{\mathrm{pivot}}$.*

Proof of Proposition 5. Set $a = t_{K_{\mathrm{pivot}}}$ and $b = t_k$ in Lemma 1. The corrected score is $s_{t_k}(q,v)$, so (6) gives

$$E_k(q) = \{v : s_{t_k}(q,v) \ge \tfrac{1}{2} \cdot \|q\|^2\} = A_k(q).$$

Every vertex in $S_k(q)$ enters the FIFO. Along an envelope path $s = v_0, v_1, \ldots, v_h = v$. Expansion of v_i visits v_{i+1}. Its

exact envelope test succeeds, so it is also enqueued. Induction and exhaustive FIFO expansion give $E_k(q) \subseteq C_k(q)$. Exact verification then returns $C_k(q) \cap A_k(q) = A_k(q)$. Whether a finite graph satisfies the reachability premise is measured empirically, not guaranteed by this proposition. □

I. Single-request latency and batched throughput

Table IX separates isolated request latency from the batched throughput reported in the main evaluation. The left block uses one query, one thread, and one decode. The right block reports per-query time with 64 threads at the largest measured batch, which is 100 for Music100 and 1,000 for the other datasets. Each entry is the median of three independent process repetitions. Since the two blocks change concurrency as well as batch size, their ratio is not a pure batching speedup.

TABLE IX
SINGLE-REQUEST AND BATCHED PER-QUERY TIME (MS).

	One-thread single request			64-thread largest batch / query		
Dataset	k=10	50	100	k=10	50	100
SIFT1M	17	17	17	0.017	0.021	0.053
DEEP1M	17	17	17	0.017	0.021	0.023
GIST1M	25	25	30	0.35	0.44	0.75
Music100	18	21	28	0.076	0.22	0.23
MSMarco1M	70	52	65	1.5	1.1	1.6

J. Experimental configurations

The submission states the common configuration and the budget policy. This section records the dataset-specific values used by the reported results. All ANQI rows use approximate-AKNN thresholds and M8+u8. Exact thresholds are used only for evaluation, except for the standalone Local Radius Law diagnostic described in Section V-E. Complete sweep grids and launch commands are available in the implementation repository.

For the million-scale benchmark, the shared index configuration is $G = 48$, $K_{\text{pivot}} = 100$, and $J = 6$. The AKNN source settings are listed in Table X. "Refine" is the number of NN-Descent refinement rounds.

TABLE X
MILLION-SCALE APPROXIMATE-AKNN SOURCE SETTINGS.

Dataset	Source k	Leaf	Trees	EF	Refine
SIFT1M	100	500	64	600	0
DEEP1M	100	1000	128	800	0
GIST1M	100	1000	64	800	8
Music100	100	5000	128	800	0
MSMarco1M	100	1000	64	400	0

Table XI gives the search budget of each operating point in Table IV. The full curves vary L around these values while holding the constructed index fixed.

TABLE XI
SELECTED MILLION-SCALE ANQI SEARCH BUDGETS.

Dataset	$k = 10$	$k = 50$	$k = 100$
SIFT1M	20	52	85
DEEP1M	40	80	100
GIST1M	320	480	640
Music100	800	3800	3800
MSMarco1M	2200	1500	2200

The scale experiment materializes only $k = 10$ and uses $K_{\text{pivot}} = 10$, one LocalJoin pass, $\beta = 0.5$, and a degree-20 graph. SIFT10M uses an approximate top-100 source with leaf size 500, 128 trees, and EF 1200. SIFT100M and SIFT1B use precomputed approximate top-10 streams. The selected search settings are EF 512, 1152, and 768 at 10M, 100M, and 1B. The verifier slack is zero at 10M and 100M and 0.001 at 1B. The 1B graph is expanded to degree 20 from a width-12 core. Its validation curve and fresh audit each use 100,000 queries.

K. Computation and storage complexity

Let T_{AKNN}, T_{LRQ}, and $T_{\mathcal{E}}$ denote AKNN, LRQ, and entry construction. If graph construction evaluates E_b pairs and performs P_b width-B_b queue updates, its total cost is

$$O\Big(T_{\text{AKNN}} + T_{\text{LRQ}} + T_{\mathcal{E}} + E_b \cdot D + P_b \cdot \log B_b + n \cdot \big[D + |\mathcal{K}| \cdot (M + b)\big]\Big). \tag{29}$$

This includes threshold generation, model fitting, lifting, refinement, and pruning. The reported build time also includes persistence.

For the compact implementation, excluding the shared base vectors and construction temporaries, index size is

$$S_{\text{index}} = O\Big(n \cdot (G + M + 2) + |\mathcal{K}| \cdot M + |\mathcal{K}| \cdot 2^b\Big) + n \cdot |\mathcal{K}| \cdot b \text{ bits}. \tag{30}$$

The word term counts adjacency, row metadata, one route cutoff, coefficients, model state, codewords, and the absorbed linear entry structure. For fixed G, M, $|\mathcal{K}|$, and b the state per object is bounded, unlike materialized reverse lists (Section II). With $G = 48$, $M = 8$, $|\mathcal{K}| = 100$, and $b = 8$ it is about 332 bytes per object, to which the evaluation implementation adds the lifted vectors counted in Table VI. Decoding costs $O(n \cdot M)$ time and one transient n-scalar column per batch.

For one query, let $S_{\text{adj}}(q)$ be the number of adjacency entries scanned. The online cost is

$$O(E_{\text{graph}}(q) \cdot D + S_{\text{adj}}(q) + E_{\text{graph}}(q) \cdot \log L + |C(q)|)\,. \tag{31}$$

The last term is scalar threshold verification because traversal caches each query–object score. Decoding amortizes to $O(n \cdot M/|\mathcal{Q}|)$ per query. A worker uses $O(n + L + |C(q)|)$ state. The worst case is a linear scan, but lifted routing is intended to keep both graph work and $|C(q)|$ small.

L. Implementation details

Up to SIFT10M, APPROXKNN uses tree-seeded beam gathering. The SIFT100M and SIFT1B profiles import the same

logical outputs from precomputed streams whose generation time is included in the reported build time. Both paths then reweight the IDs, run the configured LocalJoin refinement, freeze $\mathcal{H}$, reverse-gather, and destination-prune in lifted space. BUILDENTRYSTRUCTURE selects the entry state from the construction sample, and FITANDENCODELRQ uses the sampling, alternating fit, endpoint extension, and floor encoding of eq. (16).

At query time, Algorithm 2 uses two queues: the bounded queue supplies descent, while the FIFO follows a connected flood envelope without a top-L answer cap. LIFTEDROUTEKEY reproduces the ordering of (11) from the cached base-space score and $\widetilde{t}_{K_{\mathrm{pivot}}}(v)$, without storing $\bar{v}$. The configured entry seeds and construction-pivot envelope serve $k \le K_{\mathrm{pivot}}$, and the decoded cutoff filters candidates. The resource-bounded L_2 profile scores a fixed entry pool, selects a bounded seed set, and charges every pool score. Its operating cutoff and seed count are selected on validation data and then fixed. The candidate envelope uses this cutoff, while the flood envelope bridges toward the persisted route cutoff to preserve connectivity. BASESCORE computes the exact query–object score once per visited object and compares it with a transient decoded cutoff, never an exact-threshold table, and version stamps avoid clearing seen per query.